lsqvce

Least-squares adjustment with variance component estimation

Theory, practical considerations, implementation, and error handling

— with the Review R5 of toolbox v4.0 and the redesign plan for v5

Prepared for the `lsqvce` toolbox (`matweigelt/lsqvce`) by a three-expert panel:

Expert A least-squares adjustment and stochastic error modelling
Expert B satellite-gravimetry data processing (GRACE/GRACE-FO)
Expert C software design and MATLAB programming

Version 5.0-plan, 1 September 2026 — the R5 record; implementation authority: the Manual

Created 2026-09-01 (CEST, UTC+02:00) with assistance from Claude (Anthropic), model Claude Fable 5.1 — Review R5. Source: `dev/docs_src/theory_r5/`. Binding status and ledger: `dev/HANDOVER.md` §12.

How to read this document

Parts I–III are the *theory authority* of the toolbox: what is computed, why, and where the computation must refuse. They supersede `lsqvce_theory.pdf` (R4 WP-59) once decision R5-D6 is taken. Parts IV–V are the review of the tree at `main` head `d7ad051` (v4.0, 634 test points, 83 public names) and the plan for v5. Every claim about the toolbox names a file, a line or a work package so it can be checked rather than believed. Nothing in Part V is implemented; it is a plan awaiting the user's decisions (§18.2).

Where this record stops (added 2026-09-09, R6 WP-106)

Parts IV–V and Appendix C are a DATED RECORD: the review of v4.0 and the plan and option mapping of v5.0 as of 1–3 September 2026. They are kept as written. Every implementation-facing statement in them — the `epochCovariance` hook, `arcId`, the `lsqvce:fit:notYet` refusal of a robust pass on the streaming kind, `LogFile`, the 14-knob option table — has since been superseded, and the current state is NOT in this document: the *implementation authority* is the Manual (`doc lsqvce`, chapters 3, 4 and 7), `RELEASE_NOTES.md` lists what changed per version (5.1 … 6.1), and `dev/HANDOVER.md` §12 carries the decisions. Parts I–III (the theory) are current.

Contents

I Theory **3**

1 The linear model and its solutions **4**

1.1 Gauss–Markov model 4

1.2 Decorrelation and whitening 4

1.3 Two ways to solve, and why the toolbox keeps both 4

1.4 Block-arrow systems and the elimination of local parameters 5

1.5 Datum defect, prior information and regularisation 5

1.6 Nonlinear models 6

1.7 What the estimates mean, and what they do not 6

2 Stochastic modelling of correlated observations **7**

2.1 Stationary noise and the Toeplitz covariance 7

2.2 Covariance function and power spectral density 7

2.3 Whitening an arc 7

2.4 Multi-axis streams 8

2.5 Where a declared law can be wrong 9

3 Variance component estimation **10**

3.1 The problem 10

3.2 The Helmert estimator and its simplified form 10

3.3 The redundancy is the hard part 11

3.4 Estimating the covariance function: the per-lag components 11

3.5 Arc-wise variance factors 12

3.6 Group weights between observation types and priors 12

3.7 Uncertainty of the components 12

3.8 Regularisation weight by VCE 12

3.9 Convergence, identifiability, and what the data cannot resolve 13

4 Quality assessment: testing the model, localising the fault **14**

4.1 The five levels of a verdict 14

4.2 Whitened residuals, redundancy and leverage 14

4.3 Data snooping and its robust cousin 14

4.4 Reliability 15

4.5 Systematic errors and whiteness 15

4.6 Parameter-level diagnostics 15

II Practical considerations **16**

5 The processing context and its numbers **17**

5.1 What a GRACE-FO month looks like to the adjustment . . . 17
5.2 The workflow, step by step . . . 17
5.3 Determinism and reproducibility . . . 18
5.4 Parallelism: which axes, and what does not parallelise . . . 18
5.5 Numerical considerations that decide correctness . . . 18
5.6 Data handling: the builder contract, caching, and what the toolbox owns . . . 19

III Implementation and error handling **20**

6 Anatomy of the computation **21**

6.1 One loop, three update policies . . . 21
6.2 The pass record . . . 21
6.3 Collectors in the sweep . . . 21
6.4 The accumulator . . . 21
6.5 Layers and the call rule . . . 22

7 Error handling **23**

7.1 Three outcomes and no fourth . . . 23
7.2 Identifiers . . . 23
7.3 Refusal catalogue: what is refused, and with what evidence . . . 23
7.4 Skips are results . . . 24
7.5 Warnings . . . 24
7.6 Convergence is a result . . . 24
7.7 User functions . . . 25
7.8 Parallel execution . . . 25
7.9 What the tests must prove . . . 25

IV Review R5 of `lsqvce` v4.0 **26**

8 Method, scope, and what is right **28**

8.1 Method . . . 28
8.2 What R5 confirms as right . . . 28

9 Findings H-01 . . . H-14 **29**

10 Limitations of the method (to document, not to fix) **32**

11 The panel's discussion **33**

V The redesign: `lsqvce` v5 **34**

12 Design principles and the logic **35**
12.1 The logic, top down . . . 35

13 The user interface **36**
13.1 Three lines, all kinds . . . 36
13.2 `lsqvce.fit` . . . 36
13.3 `lsqvce.Options` — twelve properties and four groups . . . 36
13.4 The data contract, v5 delta . . . 37
13.5 `lsqvce.Results` . . . 37

14 The function set **38**

15 Workflow, data handling, parallelism **41**

16 Tests **42**
16.1 Categories (seven, counted per unit) and tags . . . 42
16.2 The oracle register . . . 42
16.3 Per-unit matrix (what each category asserts) . . . 42
16.4 Gates . . . 43

17 Documentation **44**

18 Work packages, levels, decisions **45**
18.1 Levels M0 . . . M7 . . . 45
18.2 Decisions required . . . 47
18.3 Risks and how the plan bounds them . . . 47
18.4 What does not change . . . 47

A Old → new mapping (83 → 46 public names) **48**

B Error identifier catalogue (v5 authority, initial) **50**

C Options mapping v4 → v5 (as implemented, M5) **51**

D References **53**

Part I

Theory

1 The linear model and its solutions

1.1 Gauss–Markov model

The toolbox estimates u unknowns $\boldsymbol{x}$ from n *reduced* observations $\boldsymbol{l}$ (observed minus computed at the linearisation point) under the linear Gauss–Markov model

$$\boldsymbol{l} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{e}, \qquad \mathrm{E}[\boldsymbol{e}] = \boldsymbol{0}, \qquad \mathrm{E}[\boldsymbol{e}\boldsymbol{e}^{\mathsf{T}}] = \boldsymbol{\Sigma} = \sigma_0^2 \boldsymbol{Q}, \qquad \boldsymbol{P} = \boldsymbol{Q}^{-1}. \tag{1.1}$$

$\boldsymbol{Q}$ is the a-priori cofactor matrix, σ_0^2 the variance of unit weight. The best linear unbiased estimate minimises $\boldsymbol{e}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{e}$:

$$\hat{\boldsymbol{x}} = (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{P}\boldsymbol{l} = \boldsymbol{N}^{-1}\boldsymbol{n}, \qquad \hat{\boldsymbol{v}} = \boldsymbol{l} - \boldsymbol{A}\hat{\boldsymbol{x}}, \qquad \hat{\sigma}_0^2 = \frac{\hat{\boldsymbol{v}}^{\mathsf{T}}\boldsymbol{P}\hat{\boldsymbol{v}}}{n-u}, \qquad \boldsymbol{\Sigma}_{\hat{\boldsymbol{x}}\hat{\boldsymbol{x}}} = \hat{\sigma}_0^2 \boldsymbol{N}^{-1}. \tag{1.2}$$

Two facts used everywhere below: the residual is a projection, $\hat{\boldsymbol{v}} = (\boldsymbol{I} - \boldsymbol{A}\boldsymbol{N}^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{P})\,\boldsymbol{l} = (\boldsymbol{I} - \boldsymbol{H})\,\boldsymbol{l}$ with the *hat matrix* $\boldsymbol{H}$; and the redundancy is distributed over the observations by the diagonal of $\boldsymbol{R} = \boldsymbol{I} - \boldsymbol{H}$, $\sum_i r_i = n - u$ (Förstner 1979). $\hat{\sigma}_0^2$ is unbiased only because the denominator is that trace: every estimator in Chapter 3 is a specialisation of "quadratic form over its expectation".

1.2 Decorrelation and whitening

With $\boldsymbol{\Sigma} = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{W}$ (any square root; Cholesky $\boldsymbol{W}$ upper triangular, or a filter), the model is transformed to unit weights,

$$\bar{\boldsymbol{A}} = \boldsymbol{W}^{-\mathsf{T}}\boldsymbol{A}, \qquad \bar{\boldsymbol{l}} = \boldsymbol{W}^{-\mathsf{T}}\boldsymbol{l}, \qquad \bar{\boldsymbol{e}} \sim (\boldsymbol{0}, \boldsymbol{I}), \tag{1.3}$$

and (1.2) becomes ordinary least squares. The toolbox never forms $\boldsymbol{\Sigma}$ or $\boldsymbol{P}$ of size n: whitening is applied *per arc* by a triangular solve (Toeplitz Cholesky) or by a linear filter (AR/ARMA), Chapter 2. Everything downstream — normal equations, residuals, redundancies — is expressed in whitened quantities, so "residual" in this document means $\bar{\boldsymbol{v}} = \boldsymbol{W}^{-\mathsf{T}}\hat{\boldsymbol{v}}$ unless said otherwise. The raw residual is recovered as $\hat{\boldsymbol{v}} = \boldsymbol{W}^{\mathsf{T}}\bar{\boldsymbol{v}}$ when a user asks for it.

1.3 Two ways to solve, and why the toolbox keeps both

Normal equations (NEQ). Accumulate $\boldsymbol{N} = \sum_k \bar{\boldsymbol{A}}_k^{\mathsf{T}}\bar{\boldsymbol{A}}_k$, $\boldsymbol{n} = \sum_k \bar{\boldsymbol{A}}_k^{\mathsf{T}}\bar{\boldsymbol{l}}_k$ and $\bar{\boldsymbol{l}}^{\mathsf{T}}\bar{\boldsymbol{l}}$ over arcs k, then $\boldsymbol{N} = \boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}$ (Cholesky) and two triangular solves. Cost $O(nu^2 + u^3/3)$, memory $O(u^2)$, and — decisive in satellite geodesy — *partial systems add*: arcs, days, satellites, and archived normal equations (SINEX/GROOPS) combine by matrix addition. The price is conditioning: $\kappa(\boldsymbol{N}) = \kappa(\bar{\boldsymbol{A}})^2$, so $\hat{\sigma}_0^2$ computed as $(\bar{\boldsymbol{l}}^{\mathsf{T}}\bar{\boldsymbol{l}} - \boldsymbol{n}^{\mathsf{T}}\hat{\boldsymbol{x}})/(n-u)$ cancels catastrophically when the fit is very good relative to $\|\boldsymbol{l}\|$ (§5.5).

Square-root information (QR). Maintain the triangular factor of $[\bar{\boldsymbol{A}}\ \bar{\boldsymbol{l}}]$ by Householder updates; $\hat{\sigma}_0^2$ is the squared last diagonal, obtained *without* subtraction. Backward stable, $\kappa(\bar{\boldsymbol{A}})$ instead of its square; memory the same $O(u^2)$; arcs still add (the factor is updated, not recomputed); but factors of two independent systems do not add — combination means re-triangularising the stacked factors, which is exact but not the file-exchange convention of the field.

Verdict (R4 §11.2 item 2, confirmed by R5)

NEQ is the default because the block-arrow elimination, day-wise parallel accumulation and NEQ file exchange are NEQ operations; QR is the accuracy fallback. Both stay. The condition evidence (κ estimate, pivot ratios) is always reported so a user sees when the default is the wrong choice. What the toolbox must not do is hide the difference by iterative refinement on the NEQ path ("guards are not fixes", HANDOVER §11.8).

1.4 Block-arrow systems and the elimination of local parameters

In satellite gravimetry the unknowns split into *common* parameters $\boldsymbol{x}$ (Stokes coefficients, shared by all arcs of all systems) and *local* parameters $\boldsymbol{y}_a$ (initial state, accelerometer scale/bias, empirical KBR terms) that appear in one arc, one day or one system only. For system s and arc a,

$$\bar{\boldsymbol{l}}_a = \bar{\boldsymbol{A}}_a \boldsymbol{x} + \bar{\boldsymbol{B}}_a \boldsymbol{y}_a + \bar{\boldsymbol{e}}_a, \tag{1.4}$$

and the joint normal equations have the block-arrow form. Eliminating $\boldsymbol{y}_a$ by the Schur complement gives the *reduced* contribution of the arc to the common system:

$$\boldsymbol{N}_a^{\mathrm{red}} = \bar{\boldsymbol{A}}_a^{\mathsf{T}} \bar{\boldsymbol{A}}_a - \bar{\boldsymbol{A}}_a^{\mathsf{T}} \bar{\boldsymbol{B}}_a \, (\bar{\boldsymbol{B}}_a^{\mathsf{T}} \bar{\boldsymbol{B}}_a)^{-1} \bar{\boldsymbol{B}}_a^{\mathsf{T}} \bar{\boldsymbol{A}}_a = \bar{\boldsymbol{A}}_a^{\mathsf{T}} \boldsymbol{P}_{B_a}^{\perp} \bar{\boldsymbol{A}}_a, \qquad \boldsymbol{n}_a^{\mathrm{red}} = \bar{\boldsymbol{A}}_a^{\mathsf{T}} \boldsymbol{P}_{B_a}^{\perp} \bar{\boldsymbol{l}}_a, \tag{1.5}$$

with $\boldsymbol{P}_{\bar{B}}^{\perp} = \boldsymbol{I} - \bar{\boldsymbol{B}}(\bar{\boldsymbol{B}}^{\mathsf{T}}\bar{\boldsymbol{B}})^{-1}\bar{\boldsymbol{B}}^{\mathsf{T}}$ the projector onto the complement of the local design. The residual sum of squares reduces likewise, $\bar{\boldsymbol{l}}_a^{\mathsf{T}} \boldsymbol{P}_{B_a}^{\perp} \bar{\boldsymbol{l}}_a$. Back-substitution recovers $\hat{\boldsymbol{y}}_a = (\bar{\boldsymbol{B}}_a^{\mathsf{T}}\bar{\boldsymbol{B}}_a)^{-1}\bar{\boldsymbol{B}}_a^{\mathsf{T}}(\bar{\boldsymbol{l}}_a - \bar{\boldsymbol{A}}_a\hat{\boldsymbol{x}})$ and, if wanted, its covariance. Three consequences shape the toolbox:

1. The common system is assembled from *arc contributions that are independent of each other*: the accumulate step is embarrassingly parallel over arcs, days and systems (Chapter 5).
2. Scopes above the arc (a day-wise accelerometer bias, a system-wise phase-centre offset) are still local to their scope and are eliminated at that scope; the reduced contribution of a day is the Schur complement of the day's local block over the day's arcs. The v4 *scoped parameter tables* (HANDOVER L11) are exactly this bookkeeping.
3. Every diagnostic that involves the hat matrix (redundancy, leverage, w-tests) must include the local block's projector: $\boldsymbol{H}_a = \boldsymbol{P}_{B_a} + \boldsymbol{P}_{B_a}^{\perp} \bar{\boldsymbol{A}}_a \boldsymbol{N}^{-1} \bar{\boldsymbol{A}}_a^{\mathsf{T}} \boldsymbol{P}_{B_a}^{\perp}$. Forgetting the first term over-states the redundancy by n_l per arc (R2 finding F-03, fixed WP-33).

1.5 Datum defect, prior information and regularisation

A rank defect $d = u - \mathrm{rank}(\boldsymbol{A})$ (a datum that no observation fixes; a truncation degree the data cannot resolve) is detected from the trailing pivots of the Cholesky factor or from an eigenvalue ratio, never from a factoriser's success/failure at the boundary (R4 §11.8: regime-dependent). It is removed by either

- *minimum constraints* $\boldsymbol{B}^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{0}$ with $\boldsymbol{B}$ a basis of the null space, or
- *pseudo-observations* $\boldsymbol{0} = \boldsymbol{L}\boldsymbol{x} + \boldsymbol{e}_r$, $\boldsymbol{e}_r \sim (\boldsymbol{0}, \sigma_r^2 \boldsymbol{Q}_r)$, which is Tikhonov regularisation with $\lambda = \hat{\sigma}_0^2/\sigma_r^2$: $\boldsymbol{N}_\lambda = \boldsymbol{N} + \lambda \boldsymbol{L}^{\mathsf{T}} \boldsymbol{Q}_r^{-1} \boldsymbol{L}$.

The pseudo-observation form is the one that fits the VCE machinery: the prior is one more observation group with its own variance component σ_r^2, estimated by Helmert like any other (Koch & Kusche 2002; Ellmer 2018 §6.5.5: the "geophysical pseudo-observations" of ITSG-Grace2016). The toolbox's rule is that regularisation is always *declared* (a group with a name, a matrix and a component) and always *reported* (its redundancy share, its component, its effect on $\hat{\sigma}_0^2$), never applied silently to make a factorisation succeed.

1.6 Nonlinear models

For $\boldsymbol{l} = f(\boldsymbol{x}) + \boldsymbol{e}$ the toolbox iterates Gauss–Newton on the linearised model $\Delta\boldsymbol{l}^{(k)} = \boldsymbol{l} - f(\boldsymbol{x}^{(k)}) = \boldsymbol{A}(\boldsymbol{x}^{(k)})\,\Delta\boldsymbol{x} + \boldsymbol{e}$, with an optional Levenberg–Marquardt damping $\lambda\,\mathrm{diag}(\boldsymbol{N})$ that is a *step control*, not a prior: the damping must be reported and must vanish at convergence, or the covariance $\boldsymbol{N}^{-1}$ is that of a different problem. In the streaming design the builder receives the current $\boldsymbol{x}^{(k)}$ and returns the re-linearised arc; the outer loop is the same loop as the VCE loop (Chapter 6), which is the reason the redesign folds "nonlinear", "robust" and "VCE" into one iteration with three update policies.

1.7 What the estimates mean, and what they do not

$\boldsymbol{\Sigma}_{\hat{x}\hat{x}} = \hat{\sigma}_0^2 \boldsymbol{N}^{-1}$ is the *formal* error: correct if the functional model is complete, the stochastic model is right, and the residuals are white. Chapter 3 makes the second condition true as far as the data allow; Chapter 4 tests the first and the third. An error bar without those tests is a claim, not a measurement — which is why the toolbox couples every fit to a verdict (§4.1).

2 Stochastic modelling of correlated observations

2.1 Stationary noise and the Toeplitz covariance

Instrument noise on a satellite stream (KBR range-rate, LRI, kinematic positions, accelerometer-derived quantities) is coloured: its power spectral density (PSD) is far from flat, typically rising towards low frequencies. Within one arc of m equally spaced epochs (Δt) the noise is modelled as a wide-sense stationary process with autocovariance function $C(\tau) = C(|t_i - t_j|)$ (Ellmer 2018 §6.5.1). The arc covariance is then symmetric Toeplitz,

$$\boldsymbol{\Sigma} = \sigma^2 \sum_{n=0}^{m-1} c_n \boldsymbol{V}_n, \qquad c_n = C(n\Delta t)/C(0) \text{ or unnormalised}, \qquad (\boldsymbol{V}_n)_{ij} = \delta_{|i-j|,n}, \tag{2.1}$$

where $\boldsymbol{V}_n$ is the n-th *covariance component* matrix (identity for $n = 0$, ones on the n-th sub- and super-diagonal otherwise; eq. 6.5.16 of Ellmer). Two facts matter for implementation: the decomposition (2.1) is *linear in the lag values*, which is what makes VCE per lag possible (Chapter 3); and a Toeplitz matrix is positive definite if and only if the PSD it corresponds to is positive at every frequency (Bochner), which is where *truncated* or *noisy* covariance functions fail (§2.5).

Stationarity is assumed per arc and per month; the covariance function is estimated once per observation type and month (Ellmer). Deviations from stationarity within a month are absorbed by the arc-wise variance factors (§3.5) — a scalar per arc, which is all the data can resolve at that scale.

2.2 Covariance function and power spectral density

For a discrete stationary sequence the one-sided PSD $S(f)$ and the autocovariance c_n form a cosine-transform pair (Wiener–Khinchin). On a grid of N lags and N frequencies the pair is the *type-I DCT* with its own inverse up to scale,

$$S_j = \sum_{n=0}^{N-1} w_n c_n \cos\frac{\pi jn}{N-1}, \qquad c_n = \frac{2}{N-1}\sum_{j=0}^{N-1} w_j S_j \cos\frac{\pi jn}{N-1}, \qquad w_0 = w_{N-1} = \tfrac{1}{2},\ w_k = 1 \text{ else}, \tag{2.2}$$

with $f_j = j/(2(N-1)\Delta t)$ up to Nyquist. The toolbox's `kernel.dctI` implements this pair with two normalisations (`ortho` = self-inverse unitary; `ellmer` = the GROOPS convention). *Which normalisation is used must be pinned once and never mixed*: an amplitude estimated in one convention and transformed back in the other is off by a constant factor that VCE then "corrects" by re-estimating — convergence to the wrong fixed point (R2 finding F-04 was of this kind).

Why estimate the PSD and not the covariance function directly? A variance component is estimated as a square and cannot change sign; a covariance value at a lag can be negative, a PSD value cannot. Estimating amplitudes per *frequency* keeps every component non-negative and lets the inverse transform produce negative lag values legitimately (Ellmer 2018 §6.5.3). The same argument justifies the *floor* policy: a PSD amplitude that VCE drives to zero is clamped to a small positive fraction of its neighbours, because $S_j = 0$ is a singular covariance, not a measurement (WP-35).

2.3 Whitening an arc

Three equivalent routes produce $\bar{\boldsymbol{A}} = \boldsymbol{W}^{-\mathsf{T}}\boldsymbol{A}$ of eq. (1.3) for one arc; they differ in cost, exactness and what they need from the model.

Toeplitz Cholesky (exact). $\boldsymbol{\Sigma} = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{W}$ by a dense Cholesky, $O(m^3)$, or by the Levinson/Schur recursion, $O(m^2)$; the triangular solve is $O(m^2 u)$ for u columns. Exact for any PD Toeplitz law given by its lags. This is the route of GROOPS and of `engine.covModel` for `type = 'acf'|'psd'`. Its failure mode is a non-PD law (§2.5).

AR/ARMA filter. If the noise is (approximately) an AR(p) process, $e_t + a_1 e_{t-1} + \cdots + a_p e_{t-p} = \eta_t$, η white with variance σ_η^2, then whitening is the FIR filter $\boldsymbol{W}^{-\mathsf{T}}\boldsymbol{x} \approx (\boldsymbol{x} * \boldsymbol{a})/\sigma_\eta$, cost $O(mpu)$, and *no covariance matrix is ever formed*. The AR coefficients are obtained from the covariance function by the Levinson–Durbin recursion (Yule–Walker), and a high-order AR can be reduced to a compact ARMA(p,q) by Durbin's method (`kernel.ar2arma`). The approximation is in the *start-up*: the first p samples of an arc are not correctly whitened by a causal filter unless the exact initial-state weights are applied (the toolbox applies the exact Cholesky rows for the first p samples, `kernel.armaWhiten`; residual approximation zero). The filter route is the right choice when arcs are long ($m \gg p$) and the PSD is smooth; the Toeplitz route when the PSD has narrow features (1/rev harmonics, the KBR "bump") that a low-order AR cannot represent. The reduction quality is checked by re-deriving the ACV of the fitted AR and comparing it with the target (`engine.checkPSD`); this instrument must be part of the estimation, not an optional utility (Part IV).

Batching within an arc. A long arc may be processed in row batches; a filter has memory, so batches must overlap by $\max(p,q)$ samples and the overlap rows are discarded from the second batch. Batching never changes the result (pinned by test; DATA_CONTRACT §5).

2.4 Multi-axis streams

Kinematic positions carry $d = 3$ components per epoch with cross-axis correlation from the orbit solution. Two models are supported:

- **Separable**: $\boldsymbol{\Sigma} = \boldsymbol{R}_t \otimes \boldsymbol{D}$ with a temporal law $\boldsymbol{R}_t$ (Toeplitz) and a per-epoch $d \times d$ matrix $\boldsymbol{D}_i$ (from the orbit's own epoch covariance). Whitening is the per-epoch $\boldsymbol{D}_i^{-1/2}$ (de-scale) followed by the temporal whitener per axis (`kernel.separableWhiten`). The per-epoch matrices must be supplied by the data (the `epochCovariance` hook of the contract); without them the fallback "one scalar per stream" *biases the temporal law* whenever $\boldsymbol{D}_i$ varies with time, because the de-scaled residuals are then not stationary (HANDOVER L10, WP-72 rationale).
- **Vector AR**: $\boldsymbol{e}_t = \sum_{k=1}^{p} \boldsymbol{A}_k \boldsymbol{e}_{t-k} + \boldsymbol{\eta}_t$ with $d \times d$ coefficient matrices, fitted by the multichannel Levinson (Whittle) recursion from the cross-ACV (`engine.whittle`, `kernel.varACV`, `kernel.varWhiten`). This is the model of Kvas (2019, §2.2) for the process noise of daily fields; for observation noise it is the exact treatment of cross-axis correlation and is the one the redesign keeps as the multi-axis option.

2.5 Where a declared law can be wrong

Non-positive-definite covariance laws

A covariance function that is *truncated* at lag L (zero beyond) is in general *not* positive definite: its spectrum is the true PSD convolved with a Dirichlet kernel, which has negative side lobes. The toolbox found this in production (R4 finding G-08): an `acf` handed over with L shorter than the longest arc produced an indefinite Toeplitz matrix, every arc was skipped as non-PD, and the driver reported "no arcs". The correct treatments, in order of preference: (i) define the law by its PSD and transform (positivity is then automatic); (ii) taper the covariance function with a PD window (Bartlett/Parzen) rather than truncating; (iii) require $L \geq m_{\max}$ and refuse otherwise. The toolbox does (iii) today; the redesign does (i) as the internal representation — the estimator works in the PSD domain anyway — and accepts (ii) as a user option.

Sampling and units. A covariance function is a function of *time*; a lag index is a function of *samples*. The two coincide only for $\Delta t = 1\,\mathrm{s}$. The G-08 defect above was a lookup in samples of a law given in seconds. The internal representation of every law must carry its Δt (R2: metadata travels with the data), and any resampling is explicit.

Gaps inside an arc. The Toeplitz structure assumes equal spacing. A gap is either a cut (two arcs) or an explicit missing-data treatment (the covariance of the remaining epochs is the Toeplitz matrix with the gap rows/columns removed — still valid, no longer Toeplitz, $O(m^3)$). The toolbox cuts; the data contract must make the cut a declared arc boundary (the arc resolver), never an implicit one.

3 Variance component estimation

3.1 The problem

The stochastic model is known up to a finite set of parameters $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_G)$ entering linearly,

$$\boldsymbol{\Sigma}(\boldsymbol{\theta}) = \sum_{g=1}^{G} \theta_g \boldsymbol{V}_g, \tag{3.1}$$

where the $\boldsymbol{V}_g$ are known symmetric matrices. Three instances cover everything the toolbox does:

(a) **Group weights.** $\boldsymbol{V}_g$ is block-diagonal with the a-priori cofactor $\boldsymbol{Q}_g$ of observation group g (an observation type, a satellite, an arc, a prior) and zeros elsewhere: $\theta_g = \sigma_g^2$ is the variance factor of the group. Koch (1999), Koch & Kusche (2002).

(b) **Covariance function per lag.** Within one group, $\boldsymbol{V}_n$ are the Toeplitz components of eq. (2.1): $\theta_n = c_n$ (Ellmer 2018 §6.5.3). Transformed by the DCT, the amplitudes per frequency S_j are an equivalent parameterisation with a positivity constraint.

(c) **Arc-wise factors.** $\boldsymbol{V}_a$ is the group's full Toeplitz law restricted to arc a: $\theta_a = \sigma_a^2$ (Ellmer §6.5.4).

The three are nested and are estimated together in the ITSG scheme; the toolbox's "ellmer" model is exactly this. The sections below derive one estimator and show that all of the toolbox's variants are its specialisations.

3.2 The Helmert estimator and its simplified form

The quadratic form $\hat{\boldsymbol{v}}^\mathsf{T}\boldsymbol{P}\boldsymbol{V}_g\boldsymbol{P}\hat{\boldsymbol{v}}$ has expectation (Koch 1999, §3.6)

$$\mathrm{E}[\hat{\boldsymbol{v}}^\mathsf{T}\boldsymbol{P}\boldsymbol{V}_g\boldsymbol{P}\hat{\boldsymbol{v}}] = \sum_h \theta_h \operatorname{tr}(\boldsymbol{R}\boldsymbol{V}_g\boldsymbol{R}\boldsymbol{V}_h) \quad \text{with} \quad \boldsymbol{R} = \boldsymbol{P} - \boldsymbol{P}\boldsymbol{A}\boldsymbol{N}^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{P} \tag{3.2}$$

($\boldsymbol{R}$ here is the weighted redundancy matrix, eq. 6.5.35 of Ellmer). Setting the observed quadratic forms equal to their expectations gives the *Helmert system* $\boldsymbol{H}\boldsymbol{\theta} = \boldsymbol{q}$, $H_{gh} = \operatorname{tr}(\boldsymbol{R}\boldsymbol{V}_g\boldsymbol{R}\boldsymbol{V}_h)$, $q_g = \hat{\boldsymbol{v}}^\mathsf{T}\boldsymbol{P}\boldsymbol{V}_g\boldsymbol{P}\hat{\boldsymbol{v}}$, an unbiased estimator (it is the BIQUE/MINQUE of Rao under normality, and the LS-VCE of Teunissen & Amiri-Simkooei (2008) with a particular weight). Its cost is G^2 traces of $u \times u$ products; at G of order thousands (lags × streams × arcs) it is out of reach, and its off-diagonal blocks are small when the groups are disjoint.

The *simplified* (Förstner 1979) estimator drops the off-diagonal terms and iterates:

$$\hat{\theta}_g^{(k+1)} = \theta_g^{(k)} \frac{\Omega_g}{r_g}, \qquad \Omega_g = \bar{\boldsymbol{v}}_g^\mathsf{T}\bar{\boldsymbol{v}}_g \text{ (whitened with } \theta^{(k)}), \qquad r_g = \operatorname{tr}(\boldsymbol{R}^{(k)}\boldsymbol{V}_g) = \sum_{i \in g} r_i, \tag{3.3}$$

where r_g is the *partial redundancy* of the group. Both estimators share the fixed point $\Omega_g = r_g$ for all g (a whitened residual sum equal to its degrees of freedom); the simplified one is a fixed-point iteration on it and converges in a few passes from a badly wrong start. This is what the toolbox implements for all three instances (a)–(c), and what GROOPS implements.

The one estimator

Every variance component in `lsqvce` is "a residual quadratic form divided by its redundancy", eq. (3.3). A group factor uses Ω_g, r_g summed over the group's observations; a lag component uses $\Omega_n = \bar{\boldsymbol{v}}^\mathsf{T}\boldsymbol{V}_n\bar{\boldsymbol{v}}$ and $s_n = \operatorname{tr}(\boldsymbol{R}\boldsymbol{V}_n)$ summed over arcs; an arc factor uses the sums over all lags of one arc. The redesign therefore needs exactly one machinery — the per-arc (Ω_n, s_n) collectors — and derives all three from it.

3.3 The redundancy is the hard part

Ω_g is a sum of squares over the residual sweep and is exact and cheap. $r_g = \mathrm{tr}(\boldsymbol{R}\boldsymbol{V}_g)$ needs $\boldsymbol{N}^{-1}$. Per arc (eq. 6.5.36–6.5.38 of Ellmer, extended by the local block of §1.4):

$$s_n^{(a)} = \mathrm{tr}(\boldsymbol{\Sigma}_a^{-1}\boldsymbol{V}_n) - \mathrm{tr}\left(\boldsymbol{U}^{-\mathsf{T}}\bar{\boldsymbol{A}}_a^{\mathsf{T}}\boldsymbol{P}_{B_a}^{\perp}\tilde{\boldsymbol{V}}_n\boldsymbol{P}_{B_a}^{\perp}\bar{\boldsymbol{A}}_a\boldsymbol{U}^{-1}\right) - \mathrm{tr}\left(\boldsymbol{P}_{B_a}\tilde{\boldsymbol{V}}_n\right), \tag{3.4}$$

with $\tilde{\boldsymbol{V}}_n = \boldsymbol{W}^{-\mathsf{T}}\boldsymbol{V}_n\boldsymbol{W}^{-1}$ the component seen through the whitener and $\boldsymbol{N} = \boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}$ the Cholesky factor of the *accumulated, reduced* common system. The first trace is a closed form of the lags; the third is an $n_l \times n_l$ problem per arc and exact; the second is the expensive one — it is the trace of a $u \times u$ matrix formed from $m \times u$ products for every arc and every lag.

Exact evaluation. For moderate u the middle trace is $\|\tilde{\boldsymbol{V}}_n^{1/2}\boldsymbol{P}^{\perp}\bar{\boldsymbol{A}}_a\boldsymbol{U}^{-1}\|_F^2$, one triangular solve of $m \times u$ per arc, then per lag a shifted inner product: $O(mu^2 + Nmu)$ per arc. The toolbox's `engine.redundancy` / `kernel.redundancyLags` do this when the "exact" route is selected, and the per-observation redundancies $r_i = 1 - h_{ii}$ come out of the same sweep — which is what data snooping (§4.3) needs.

Monte-Carlo evaluation (Hutchinson 1990; Koch & Kusche 2002). For a symmetric $\boldsymbol{X}$ and random $\boldsymbol{z}$ with independent ± 1 entries, $\mathrm{E}[\boldsymbol{z}^{\mathsf{T}}\boldsymbol{X}\boldsymbol{z}] = \mathrm{tr}\boldsymbol{X}$ and $\mathrm{Var}[\boldsymbol{z}^{\mathsf{T}}\boldsymbol{X}\boldsymbol{z}] = 2\sum_{i\neq j}X_{ij}^2 \le 2\|\boldsymbol{X}\|_F^2$. With Z probes stacked in $\bar{\boldsymbol{Z}} = \boldsymbol{Z}/\sqrt{Z}$,

$$\mathrm{tr}(\boldsymbol{X}) \approx \mathrm{tr}(\bar{\boldsymbol{Z}}^{\mathsf{T}}\boldsymbol{X}\bar{\boldsymbol{Z}}), \qquad \text{so} \qquad s_n^{(a)} \approx \mathrm{tr}(\boldsymbol{\Sigma}_a^{-1}\boldsymbol{V}_n) - \Big\|\tilde{\boldsymbol{V}}_n^{1/2}\boldsymbol{P}^{\perp}\bar{\boldsymbol{A}}_a \underbrace{\boldsymbol{U}^{-1}\bar{\boldsymbol{Z}}}_{\text{once per pass}}\Big\|_F^2 - \mathrm{tr}(\boldsymbol{P}_{B_a}\tilde{\boldsymbol{V}}_n). \tag{3.5}$$

The $u \times Z$ block $\boldsymbol{U}^{-1}\bar{\boldsymbol{Z}}$ is formed once per pass and shared by every arc (Ellmer eq. 6.5.48; `engine.sharedProbe`); each arc then costs $O(muZ + NmZ)$ instead of $O(mu^2)$. With $Z = 100$ and u in the thousands this is the only affordable route, and it is the ITSG production route. Its properties, which the toolbox must expose rather than hide:

- The estimate is unbiased; its relative standard error is $\sim \sqrt{2/Z}\,\|\boldsymbol{X}\|_F / \mathrm{tr}\boldsymbol{X}$, i.e. a few percent at $Z = 100$ for well-conditioned systems and worse for nearly rank-deficient ones. Report it (the toolbox records the probe count, the seed and, on request, the estimator spread).
- The same probes must be used for every arc within a pass (they are), and the seed must be recorded in the provenance so a run is reproducible (WP-36/44).
- A per-observation redundancy r_i is *not* available from the probe route: $\boldsymbol{z}^{\mathsf{T}}\boldsymbol{X}\boldsymbol{z}$ estimates a trace, not a diagonal. Data snooping on the streaming path therefore uses the exact per-arc route or the diagonal estimator $\mathrm{diag}(\boldsymbol{X}) \approx \frac{1}{Z}\sum_z \boldsymbol{z} \odot (\boldsymbol{X}\boldsymbol{z})$ (Bekas et al. 2007), whose per-element error is much larger — acceptable for screening, not for a w-test verdict. This limitation is inherent and is stated in Chapter 4.

3.4 Estimating the covariance function: the per-lag components

Per arc a and lag $n < m_a$: $\Omega_n^{(a)} = \bar{\boldsymbol{v}}_a^{\mathsf{T}}\tilde{\boldsymbol{V}}_n\bar{\boldsymbol{v}}_a$ (a lagged sum of products of whitened residuals, computed in $O(mN)$ by shifting) and $s_n^{(a)}$ from eq. (3.4)/(3.5). Summed over the arcs of the group and transformed to frequency by the DCT (eq. 6.5.25–26),

$$\tilde{\Omega}_j = \sum_a \left(\boldsymbol{X}\boldsymbol{\Omega}^{(a)}\right)_j, \qquad \tilde{s}_j = \sum_a \left(\boldsymbol{X}\boldsymbol{s}^{(a)}\right)_j, \qquad \hat{S}_j = \frac{\tilde{\Omega}_j}{\tilde{s}_j}\,S_j^{(k)}, \tag{3.6}$$

where arcs shorter than N contribute zeros beyond their length (padding, eq. 6.5.22–23). The update is multiplicative on the previous PSD; the covariance function for the next pass is the inverse DCT of $\hat{S}$, and the whitener is rebuilt from it (Toeplitz or AR-reduced, §2.3). Three practical rules follow from the theory:

1. **Resolution.** The grid N must satisfy $N \geq m_{\max}$ or the covariance function is truncated (§2.5). The grid is derived from the schedule (the longest declared arc), never probed from data inside a `try` block.
2. **Smoothing versus redundancy.** With N frequencies and M arcs, each $\tilde{s}_j$ is of order M; an amplitude with $\tilde{s}_j$ below a threshold is not estimable and keeps its prior value (the "freeze" policy), or the spectrum is estimated on a coarser grid (Kvas 2019 §3.3: the arc length sets the spectral resolution). Smoothing the estimated PSD by a moving average trades resolution for variance and must be declared.
3. **Convergence.** The fixed-point iteration is monotone in practice but can *alternate* when the arc factors and the PSD amplitudes fight over the same overall scale (R2 finding F-15). The remedy is the Ellmer normalisation: the PSD carries the absolute level, the arc factors are normalised to unit mean per pass (§3.5), so the scale has exactly one owner.

3.5 Arc-wise variance factors

The cumulative amplitude of one arc over all frequencies, $\hat{\alpha}_a^2 = \sum_j \tilde{\Omega}_j^{(a)} / \sum_j \tilde{s}_j^{(a)}$, would double-count the level already put into the PSD. Ellmer (§6.5.4) therefore forms $\gamma_a^2 = \hat{\alpha}_a^2 \sigma_a^2$ and normalises by the mean over arcs, $\hat{\sigma}_a^2 = \gamma_a^2 / \bar{\gamma}^2$, so that the arc factors have unit mean and only re-distribute weight among arcs. Two refinements the toolbox adopts: the mean is taken *robustly* (median or trimmed) so that one destroyed arc does not shift every other factor (`MeanType = "robust"`), and a factor is clamped away from zero, with a warning naming the arc, because a factor of zero would silence the arc without any test having rejected it (WP-35).

3.6 Group weights between observation types and priors

Between streams whose *law* is already estimated by §3.4, no further group factor is needed — the PSD amplitude *is* the level. A group factor of type (a) is needed exactly for groups whose law is declared and fixed: a prior (regularisation, §1.5), a stream declared `diag` with a trusted structure, or a stream the user asks to scale only. The toolbox's separate "Scale" step (Helmert over `GroupBy` groups) and "Shape" step (temporal law) are therefore not two independent estimators but two *options* on which components are free — which is the central simplification of Part V.

3.7 Uncertainty of the components

The Fisher information of the components under normality is $\frac{1}{2}\operatorname{tr}(\boldsymbol{R}\boldsymbol{V}_g\boldsymbol{R}\boldsymbol{V}_h)$, i.e. half the Helmert matrix; its diagonal gives $\operatorname{Var}(\hat{\theta}_g) \approx 2\theta_g^2 / r_g$ for disjoint groups. A group with $r_g = 30$ has a variance factor known to $\pm 26\%$; this is why a *minimum redundancy* is enforced before a component is estimated at all, and why the toolbox reports r_g next to every component (WP-23).

3.8 Regularisation weight by VCE

With the prior as a group (§1.5), $\lambda = \hat{\sigma}_0^2 / \hat{\sigma}_r^2$ is estimated like any component: $\hat{\sigma}_r^2 = \hat{\boldsymbol{x}}^\top \boldsymbol{L}^\top \boldsymbol{Q}_r^{-1} \boldsymbol{L}\hat{\boldsymbol{x}} / r_r$ with $r_r = p - \operatorname{tr}(\boldsymbol{N}_\lambda^{-1} \lambda \boldsymbol{L}^\top \boldsymbol{Q}_r^{-1} \boldsymbol{L})$ (Koch & Kusche 2002). The alternative, generalised cross-validation, minimises a prediction-error functional and is available as an instrument (`engine.gcvLambda`); VCE is the default because it is the same estimator as everything else and its redundancy share is reported with the rest.

3.9 Convergence, identifiability, and what the data cannot resolve

- **Identifiability.** Components whose $\boldsymbol{V}_g$ are linearly dependent within the range of $\boldsymbol{R}$ are not separately estimable; the classic case is a per-arc factor and a per-day factor on the same stream. The scoped tables of the data contract prevent the user from declaring both on the same rows, and the redundancy of every component is printed.
- **Absorption.** A local parameter that fits noise (a per-arc bias on a stream with a red spectrum) absorbs low-frequency noise power; the estimated PSD then describes the noise *after* the local model, which is what the adjustment sees — correct for the weights, but not a property of the instrument. Document, do not fix.
- **Signal in the residuals.** Unmodelled geophysical signal is not noise; VCE will model it as such and down-weight exactly the arcs where the signal is strongest. This is the deepest limitation of the method (Ellmer §6.5.5 handles the modelled part by a prior with its own component). The toolbox's defence is the systematic-error test of Chapter 4 and a residual-spectrum plot the user is expected to look at.
- **Convergence criteria.** The loop stops when every free component moved by less than a tolerance ($\max_g |\hat{\theta}_g/\theta_g - 1| < \varepsilon$), when every stream passes the whiteness test, or when the pass budget is exhausted — and it reports *which*. Exhausting the budget is a result, not an error.

4 Quality assessment: testing the model, localising the fault

4.1 The five levels of a verdict

A least-squares result is judged in a fixed order, each level assuming the one below it holds (this is the toolbox's `report`, R2/R3, and it stays unchanged in v5):

0 Integrity. Does the arithmetic hold? $\sum_i r_i = n - u$ (Förstner), the whitened residual norms agree between the accumulation and the residual pass, the factorisation reproduces $\boldsymbol{N}$. A violation is a defect in the pipeline, never in the data.

1 Global test. $\hat{\sigma}_0^2/\sigma_0^2$ against $\chi^2_{n-u}/(n-u)$: is the model, as a whole, consistent with the declared noise? After VCE the test passes by construction for each group ($\Omega_g = r_g$); before VCE it is the reason to run VCE.

2 Localisation. Which observations, arcs or streams are inconsistent (§4.3); does the residual carry structure (§4.5).

3 Reliability. What could *not* have been detected: partial redundancies, minimal detectable biases, their influence on the estimates (§4.4). This is the level people skip and the one that decides whether the result can be trusted.

4 Parameters. Correlations, conditioning, contribution of each group to each parameter, confidence regions.

4.2 Whitened residuals, redundancy and leverage

With the whitened residual $\bar{\boldsymbol{v}}$ and the hat matrix $\bar{\boldsymbol{H}} = \bar{\boldsymbol{A}}\boldsymbol{N}^{-1}\bar{\boldsymbol{A}}^{\mathsf{T}}$ (plus the local projector, §1.4), $\mathrm{Var}(\bar{v}_i) = \hat{\sigma}_0^2 r_i$ with $r_i = 1 - \bar{h}_{ii}$. The *standardised* residual $w_i = \bar{v}_i/(\hat{\sigma}_0\sqrt{r_i})$ is the test statistic of everything that follows. $r_i \to 0$ means the observation is fitted regardless of its value (no control); $r_i \to 1$ means it barely influences the solution.

4.3 Data snooping and its robust cousin

Baarda's w-test assumes σ_0 known: $w_i \sim N(0,1)$; a blunder of size ∇_i shifts w_i by $\nabla_i\sqrt{r_i}/\sigma_0$.

Pope's τ-test uses $\hat{\sigma}_0$ and the τ distribution (a scaled t), appropriate when σ_0 is estimated — always the case after VCE. The toolbox chooses automatically (`Test="auto"`). With n tests the family-wise level is controlled by Bonferroni (α/n) or, less conservatively, by the largest $|w_i|$ against its extreme-value distribution; the toolbox uses Bonferroni and prints the effective threshold.

One at a time. Removing the largest $|w_i|$, re-solving, and repeating (iterative snooping) is correct but expensive on a streamed problem: every rejection is a rank-one downdate of the reduced system, which the toolbox performs on the accumulated normal equations without re-reading the data when the observation's whitened row is kept; otherwise the arc is re-read. Rejection is recorded, never silent, and the total rejected fraction is a reported number with a warning threshold.

Robust IRLS replaces the sequential rejection by a continuous down-weighting $p_i \leftarrow p_i\,\psi(w_i)/w_i$ with Huber, IGG3 (Yang 1994) or Tukey bisquare influence functions, iterated to convergence. It is the right tool when outliers are numerous or clustered; its result is *not* a least-squares result (the covariance must be scaled by the asymptotic efficiency), which the report states. In the redesign, IRLS is one of the three

update policies of the single outer loop: per-observation weights are a diagonal covariance component like any other.

Scale cross-check. The MAD of the w_i ($1.4826 \cdot \text{median}|w_i|$) estimates $\hat{\sigma}_0$ robustly; a ratio far from 1 between MAD and $\hat{\sigma}_0$ says the residual distribution has heavy tails — a warning to run IRLS before believing the snooping verdict.

4.4 Reliability

The *minimal detectable bias* of observation i at significance α and power $1-\beta$ is

$$\nabla_{0,i} = \frac{\delta_0(\alpha,\beta)\,\sigma_0}{\sqrt{r_i}}, \qquad \delta_0 = z_{1-\alpha/2} + z_{1-\beta}, \tag{4.1}$$

and its influence on the estimates is $\nabla\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\bar{\boldsymbol{a}}_i\,\nabla_{0,i}$ (external reliability). Arcs with low r_i (short arcs, many local parameters) are where undetected blunders hide; the report lists the weakest and the `RWeak` threshold flags them. On the streaming path the per-observation r_i come from the exact per-arc route (§3.3); when only the Monte-Carlo route was run, the report says "reliability not computed" rather than printing a diagonal estimate with 30% error.

4.5 Systematic errors and whiteness

After VCE the whitened residuals of every stream should be white with unit variance. Two tests are applied per stream and per pass: the Ljung–Box portmanteau $Q = m(m+2)\sum_{k=1}^{K}\hat{\rho}_k^2/(m-k) \sim \chi^2_K$ on the autocorrelation of the whitened residuals, and the largest $|\hat{\rho}_k|$ against $2/\sqrt{m}$. A stream that never whitens has a law the model class cannot represent (AR order too low; a periodic term; a non-stationary segment) and the verdict names it — the per-pass convergence table must carry the *worst stream*, not only the worst p-value (Results.convergence, WP-17).

Structure in the raw (un-whitened) residuals against time, argument of latitude, or a parameter of the user's choosing — runs tests, a low-order polynomial fit, periodogram peaks at k/rev — indicates a *functional* model error. VCE cannot repair that; it will absorb it into the noise model, at the price of over-estimated errors and down-weighted arcs. The systematic-error instrument is therefore not an optional utility a user is told to run, but part of the verdict (Part IV, finding H-05).

4.6 Parameter-level diagnostics

Correlation coefficients from $\boldsymbol{N}^{-1}$, the condition number estimate of $\bar{\boldsymbol{A}}$, the contribution of group g to parameter j ($[\boldsymbol{N}^{-1}\boldsymbol{N}_g]_{jj}$, summing to one over groups), and 2-D confidence ellipses for chosen pairs. The contribution shares are the stream-level answer to "which instrument determines this coefficient" and are cheap on the exact route, probe-estimated on the Monte-Carlo route (`engine.streamContribution`).

Part II

Practical considerations

5 The processing context and its numbers

5.1 What a GRACE-FO month looks like to the adjustment

quantity	typical value	consequence
streams	KBR/LRI range-rate, kinematic positions C and D (3 axes), optionally ACC	3–4 temporal laws to estimate
sampling	5 s (KBR), 30 s or 5 s (POD)	Δt per stream, carried with the law
arc length	1.5–3 h $\rightarrow$ $m \approx$ 1000–2000 (KBR at 5 s)	N lags of the same order
arcs	8–16 per day, $M \approx$ 300–500 per month	M is the redundancy of each PSD amplitude
common unknowns	d/o 60: $u = 3717$; d/o 96: 9405	$\boldsymbol{N}$ 100 MB–700 MB dense
local unknowns	6 ICs/arc + ACC scale/bias/drift per day + KBR empiricals	eliminated per scope
observations	$\sim 5 \cdot 10^5$ KBR, $\sim 3 \cdot 10^5$ POD rows/satellite	nothing of size n is stored
VCE passes	5–8 outer, each = accumulate + residual sweep	the run time is $\sim$ 2–3$\times$ one GLS solve per pass

The cost model per pass: accumulation $O(\sum_a m_a u^2)$, one Cholesky $O(u^3/3)$, residual/redundancy sweep $O(\sum_a m_a u Z)$ with $Z = 100$ probes, or $O(\sum_a m_a u^2)$ on the exact route. At $u \approx 4000$ the exact route is affordable on a workstation; at $u \approx 10^4$ the probe route is mandatory for the sweep, and the accumulation dominates. Memory: two $u \times u$ dense blocks (the accumulated $\boldsymbol{N}$ and its factor) plus one $u \times Z$ probe block; the per-arc work is $m \times (u + n_l)$ and is released.

5.2 The workflow, step by step

1. **Declare.** One `data` struct: which common templates (scoped), which days, which systems; each system points to one `buildDay`. Nothing is read.
2. **Validate.** Every declaration rule is checked before any read; every count (nc, nl, arcs per day) is derived from the declaration and the schedule, and is then *enforced* against every day that is read.
3. **Accumulate** (pass k): for every system, day, observation type, arc: build, cut, whiten with the current law, eliminate the local block, add to the common system. Parallel over days.
4. **Solve**: factorise the reduced common system, back-substitute the locals on request, condition evidence.
5. **Sweep**: re-read, whiten, form residuals, per-lag (Ω_n, s_n), per-observation redundancies (exact route) or probe products (MC route), whiteness statistics, worst residuals. One pass, several collectors, deterministic reduction.
6. **Update**: the PSD per stream (or pooled per observation type), arc factors, group factors for fixed-law groups, robust weights, or the linearisation point — whichever policy the run uses. Rebuild the whiteners.
7. **Gate**: converged (which criterion), not converged (why), budget exhausted. Record the pass.
8. **Judge**: the five-level verdict on the final pass; results object with provenance.

Steps 3–7 are one loop; the "gls" model is the loop with zero updates. Everything a user configures is either a declaration (step 1) or a policy of steps 6–7.

5.3 Determinism and reproducibility

Two reads of the same day must be byte-identical (the residual is formed from the second read); a reduction over days must not depend on the worker count or the order in which workers finish (fold in day order, never as-completed); the probes are seeded and the seed is recorded; the options a run actually used are stored, not the ones the user typed. A parallel run and a serial run of the same declaration agree to the reduction order's roundoff — and the test that pins this states whether the agreement is bitwise or eps-level and why (VALIDATION_GUIDE, concurrency tag).

5.4 Parallelism: which axes, and what does not parallelise

axis	parallel?	note
systems	yes	independent reduced contributions; usually only 2–3
days within a system	**yes, the axis that scales**	one raw partial per day, merged in day order (WP-62)
arcs within a day	yes	finer grain; only worth it when days are few
observation types	yes	independent whitening; small count
within an arc	no	the filter state is sequential; BLAS threads inside the dense products
the dense solve	BLAS threads only	$O(u^3)$, one per pass
probes	vectorised	one $u \times Z$ solve; no need for parallel
residual sweep	yes, over days	collectors reduced deterministically

A `parpool("threads")` is enough for the accumulation (pure MATLAB, no file I/O in the worker if the user's builder caches); a process pool is needed when the builder reads files and the I/O is the bottleneck. The toolbox decides nothing about the pool type; it decides the axis, the threshold below which a pool is not worth its overhead (measured, not guessed), and the fold order. GPU acceleration of the per-arc dense products is gated on evidence (WP-31): with $m \approx 2000$ and $u \approx 4000$ the transfer cost is comparable to the product.

5.5 Numerical considerations that decide correctness

Cancellation in $\hat{\boldsymbol{v}}^\top \boldsymbol{P} \hat{\boldsymbol{v}}$. On the NEQ path $\Omega = \bar{\boldsymbol{l}}^\top \bar{\boldsymbol{l}} - \boldsymbol{n}^\top \hat{\boldsymbol{x}}$ subtracts two numbers of size $\|\bar{\boldsymbol{l}}\|^2$ to obtain a number of size $(n-u)\hat{\sigma}_0^2$; the relative error is $\varepsilon \|\bar{\boldsymbol{l}}\|^2/\Omega$ and grows with the fit quality. When Ω comes out negative or below the cancellation floor the toolbox reports $\hat{\sigma}_0 = \mathrm{NaN}$ with one identifier, on both backends (WP-49/50); the residual sweep, which forms Ω as a sum of squares, is the remedy and supersedes the normal-form value whenever it ran. Guards, clamps or iterative refinement are not applied (R4 §11.2 item 1): they would print a number that means nothing.

Conditioning. $\kappa(\boldsymbol{N}) = \kappa(\bar{\boldsymbol{A}})^2$; the Cholesky pivots are monitored and a trailing pivot below $\varepsilon \cdot \max$ refuses with a rank-deficiency identifier that names the size of the null space. The QR backend, keeping $\kappa(\bar{\boldsymbol{A}})$, is the fallback and is what the benchmark reports when NEQ refuses (postflight L11).

Scaling. Common parameters of very different magnitude (Stokes coefficients across degrees) are column-scaled internally before factorisation and un-scaled after; the reported condition number is that of the scaled system, and the report says so.

Regime dependence. Whether a factoriser reports failure on a to-ε singular matrix depends on the BLAS/LAPACK build; assertions and refusals are formulated on eigenvalue or pivot ratios, never on the factoriser's success flag at the boundary (R4 §11.8 lesson).

5.6 Data handling: the builder contract, caching, and what the toolbox owns

The user writes one function per system, `buildDay(day, obsType, common, local, procOpt, info)`, returning raw (un-whitened) $\boldsymbol{A}$, $\boldsymbol{B}$, $\boldsymbol{l}$, $\boldsymbol{t}$ and the arc partition of the day. The toolbox owns cutting into arcs, whitening, elimination, and every count. The builder is called once per (day, observation type) and read twice per pass; the toolbox caches one built day, the user caches the expensive raw reads. `gracefoReading` (`readL1b`, `readL1bMulti`, `findGaps`) is the natural upstream: its gap/segment analysis is exactly the arc resolver the contract asks for, and its provenance struct is what `processingOptions` should carry. Nothing in `lsqvce` depends on it; a worked example that connects the two is the missing piece of the documentation (Part V).

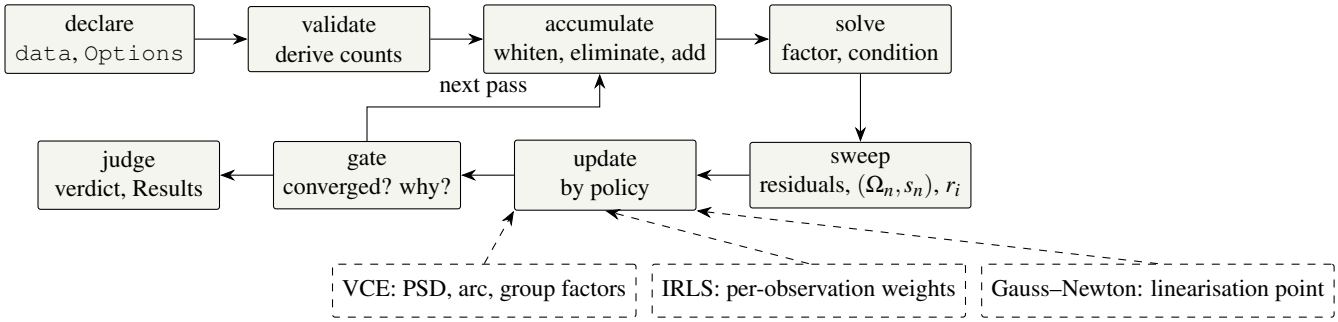
Part III

Implementation and error handling

6 Anatomy of the computation

6.1 One loop, three update policies

The v5 design has a single iteration whose body never changes; the *policy* decides what is updated between passes:



`Model="gls"` is the loop with an empty policy (one pass, judge). The policies compose: a nonlinear problem with VCE runs both updates per pass (the ITSG scheme does exactly this: re-linearised orbits and re-estimated covariance functions in the same iteration).

6.2 The pass record

Each pass leaves a row in `R.history`: the pass number, wall time per step, the update norms per component, the whiteness statistics per stream, the arcs skipped with their reason counts, the condition evidence, and the gate decision with its reason. The row is what `R.convergence()` tabulates and what a warning names when the budget is exhausted. It is written *before* the next pass starts, so an interrupted run is diagnosable.

6.3 Collectors in the sweep

The residual sweep carries a list of collectors, each with `init`, `accumulate(arc, context)` and `fold(a, b)`; the fold is associative and applied in day order. The v5 collectors: whitened residual norms per group; per-lag (Ω_n, s_n) per stream (exact or probe); per-observation r_i, w_i and the worst-K heap (exact route); the whiteness autocorrelations per stream; the group contributions. A collector that is not needed by the policy or the requested diagnostics is not attached — the sweep's cost is the sum of its collectors, and the report lists which ran.

6.4 The accumulator

One class, `engine.Accumulator` (the v5 name for today's `BlockAccumulator`; the flat batch accumulator is retired), with: a backend (NEQ or QR), a common block of size u, one store per system holding the current scope's local block and its Schur-complement machinery, `addArc`, `closeScope` (eliminate the local block of a day or a system when its last arc has been added), `solve`, `backSubstitute`, `condition`. A "batch" problem is a system with one day, one arc per batch and $n_l = 0$; a nonlinear problem re-linearises by calling the builder with the current $\boldsymbol{x}$. The class is the only place that knows the backend.

6.5 Layers and the call rule

tier	contents	calls
USER	`fit`, `Options`, `Results`, `report`	ENGINE
ENGINE	loader, accumulator, iteration, sweep, VCE, diagnostics, provenance	KERNEL, UTIL
KERNEL	transforms, filters, recursions, statistics — pure functions on arrays	UTIL
UTIL	logging, verbosity, struct helpers	—

Calls go downward only; a KERNEL function takes arrays and returns arrays, prints nothing and reads no option object. The layering is declared in one table (`tiers.csv`) and checked by the audit (check 18). A function that a USER-tier verdict tells the user to run is itself USER-tier reachable through `Results` (Part IV, H-05).

7 Error handling

Error handling in an estimation toolbox is not exception plumbing; it is the set of rules that decide *which number the user is allowed to see*. The rules below are binding for v5 and are checked mechanically where a check is possible (Part V, WP-M1).

7.1 Three outcomes and no fourth

Every operation ends in exactly one of: **refuse** (an error with a documented identifier, nothing returned), **report** (a result plus a warning naming what was assumed or skipped), or **proceed** silently (nothing was assumed). A fourth outcome — proceed silently on an assumption — is the defect class this chapter exists to remove. Corollaries:

- No function returns a numeric status code; status is either an exception or a named field of the result that the report prints.
- No empty `catch`. A `catch` either re-throws with the original as *cause* (`MException.addCause`), or converts a *named* expected failure into a recorded skip (§7.4). A bare `catch` that converts *any* error into a default also converts an undefined variable into a wrong answer.
- A `NaN` where a number belonged is the worst outcome, *except* when NaN is the honest value of a quantity that cancelled in double precision — then it is the only outcome, it carries one identifier, and the report explains it (§5.5).

7.2 Identifiers

`lsqvce:<unit>:<reason>`, unit = the function or class that raises, reason in lowerCamelCase. The documented identifier equals the raised one, enforced by a static check that parses every `error(`/`warning(` site and every help ERRORS block. The catalogue (Appendix B) is the *authority*: a new identifier is added to the catalogue in the same commit, and the audit refuses an identifier raised but not catalogued. Refusals raised on the user's behalf by the engine keep the engine's unit (`lsqvce:makeLoader:dayShape`), because that is where the evidence is; refusals of the declaration are raised by `fit` before any read.

7.3 Refusal catalogue: what is refused, and with what evidence

class	refused when	the message must carry
declaration	an unknown field, a missing required field, a scope token misuse, a duplicate id, a non-integer day, an arc id out of range	the field name, the system index/id, the rule violated (one identifier per rule)
builder output	missing `A`/`B`/`l`/`t`; row counts disagree; column counts disagree with the derived nc/nl; `arcId` does not partition	day, observation type, system, expected vs found sizes
builder threw	any exception inside the user's function	day, observation type, system; the user's exception as cause, unmodified

covariance law	non-PD Toeplitz; sigma ≤ 0; AR polynomial with roots on/inside the unit circle; law length shorter than the longest arc; Δt missing for a time-domain law	the stream, the offending lag/root/length, and the remedy (PSD form, taper, longer law)
rank	trailing pivot ratio below ε; null-space dimension $d > 0$ without a datum	d, the parameters with the smallest pivots, the backend, and the QR/regularisation remedy
estimability	a component whose redundancy is below the freeze threshold on the *first* pass with no prior value	the component key and its r_g
cancellation	Ω below the cancellation floor on the normal-form path with no residual sweep to supersede it	$\hat{\sigma}_0 = \mathrm{NaN}$ as a *result*, one identifier, the floor and both operands in the diagnostics
options	a value outside its enum/range; a combination that is contradictory (`DistributeDays` without a parallel policy; arc factors on a `diag` law)	the property, the value, the allowed set; for combinations both properties

7.4 Skips are results

An arc that cannot be whitened (non-PD law after an update), a day the resolver declares but the builder does not carry, a component that is frozen this pass — these are expected in production and are *skips*, not errors. Each skip has a reason code from a closed list, is counted per pass and per stream, and the first instance's message is kept. The pass record carries the counts; the report prints them; and a *threshold* turns a skip rate into a refusal (more than a declared fraction of arcs skipped in a stream $\Rightarrow$ the stream's law is refused, the run stops with the reason). A skip that nobody counts is the silent-degradation outcome in disguise.

7.5 Warnings

One identifier per situation; it fires *once per run*, not once per arc (the count is in the skip record); the inventory of identifiers fired is stored in `R.meta.warnings` and printed by the report. A test that provokes a documented warning disables that identifier for its duration with a cleanup-based restore; a clean run's inventory contains at most the one deliberate probe identifier (VALIDATION_GUIDE Part 5). Library code never prints: every message goes through `util.log` under one verbosity control.

7.6 Convergence is a result

Reaching the pass budget, alternating components, or a stream that never whitens are outcomes of the estimation, not failures of the code. They set `R.converged = false`, `R.stopReason` from a closed enum (`"single"`, `"converged"`, `"maxPasses"`, `"userStop"`), and one warning. The estimates of the last pass are returned with their covariance; the report's level-1 line says which criterion stopped the loop. Refusing to return a non-converged estimate would hide exactly the evidence the user needs.

7.7 User functions

The builder, the arc resolver, the epoch-covariance hook and any plot hook are user code. They are called through one wrapper that (i) checks the declared `nargin`/`nargout` *once* with `nargin(fh)`/`nargout(fh)` rather than by catching `MATLAB:TooManyOutputs` on every call (Part IV, H-08), (ii) wraps any exception with the call context and the original as cause, and (iii) validates the returned struct against the contract before it reaches the engine. A plot hook that throws is reported and disabled for the rest of the run; a builder that throws refuses the run.

7.8 Parallel execution

A worker error carries the day and system it was processing and is re-thrown on the client with that context; a pool that cannot be opened is a *report* (warning naming the reason, serial fallback), never a silent `info.fallback` string; the fold order is fixed; a serial re-run of a failing day is the documented first diagnostic step. Logging from workers is collected through one queue and emitted in day order.

7.9 What the tests must prove

Every identifier in the catalogue fires in a contract test, and only that one fires (a rejected call must not trip a different check first). Every skip reason is provoked once and shown to reach the report. The bare-`catch` count in the tree is an audit metric with a target of zero. The warning-once property is asserted with a counter over a large N. These are the `contract` and `robustness` categories of Part V applied to the error surface.

Part IV

Review R5 of `lsqvce` v4.0

Dated record

This Part and the two that follow describe the tree of 1 September 2026 (v4.0) and the v5 plan. For the implemented state see the Manual and `RELEASE_NOTES.md` (R6 WP-106).

8 Method, scope, and what is right

8.1 Method

The panel read the tree at `main` head `d7ad051` (2026-09-01: v4.0, 83 public names in `+lsqvce`: USER 4 / ENGINE 58 / KERNEL 21; 26 054 lines of package code; 23 suites, 634 declared test points, CI green; postflight green 622/622 on R2026a) with the Ellmer (2018) and Kvas (2019) theses and the `referenceSystem` guides (`CODING_GUIDE`, `VALIDATION_GUIDE`, `DOCUMENTATION_GUIDE`) open beside it, and the R2–R4 reviews of `dev/HANDOVER.md` as the record of what had already been found. Instruments: a code-only call-graph (comments and string literals stripped) rooted at the four USER names; the audit's own category matrix; a census of every `catch` block; a reading of `fit`, `Options`, `Results`, `adjustVCE`, `adjustVCEjoint`, `BlockAccumulator`, `makeLoader` and `DATA_CONTRACT.md` in full. Where a claim rests on running MATLAB (timing, numerical parity) it is marked PREDICTED; everything else is repo-verified with a file and line.

8.2 What R5 confirms as right

These stay; none of Part V touches them:

1. **The Ellmer path is complete and correct in structure.** `fit(Model="ellmer")` → `adjustVCEjoint` → `psdVCE` (eq. 6.5.19–21), `arcVarianceFactors` (6.5.27–30, robust mean, clamp), `kernel.redundancyLags` (6.5.22–23 with the local projector of WP-33), `sharedProbe`/`probeMatrix` (6.5.48, one $\boldsymbol{U}^{-1}\bar{\boldsymbol{Z}}$ per pass), `kernel.dctI` (6.5.25–26). This is the core the redesign builds on; it becomes the *only* stochastic-model estimator.
2. vTv honesty (NaN on cancellation, one identifier, both backends); NEQ default with QR fallback and visible condition evidence; the WP-36 probe contract with seeds in the provenance.
3. The v4 data contract's principle — state nothing the data already states; scope, don't number — and the block-arrow accumulator with per-scope elimination and deterministic day-order folds (WP-62).
4. Run provenance (`R.meta`), the six-category test taxonomy with the countability gate, the tolerance-provenance rule (0 unannotated tolerances), the physically structured oracle (WP-48) and the metamorphic suite (WP-29).
5. The level-based development loop, the audit as a merge gate, and the honesty invariants pinned in HANDOVER §11.2.

9 Findings H-01 . . . H-14

Each finding: evidence, verdict of the three experts, disposition (the work package of Part V that addresses it).

Design flaws and superfluous paths

H-01 Twenty-four package files are not reached by any code path from the USER surface. Code-only call graph from {`fit`, `Options`, `Results`, `report`}: unreached are `engine.Combine` (1070 lines), `BatchIndex` (470), `varComp` (503), `vceHutchinson` (379), `iterateNoiseModel` (315), `selectArOrder` (216), `PrefetchLoader` (221), `synthLoader` (257), `poolGroups`, `resolveSatSpecs`, `normalizeSatList`, `checkPSD`, `ftest`, `ellipse`, `datum`, `CommonPrior`, `snoop`, `systematic`, `kernel.crossCorr`, `toeplitzCofactor`, `covComponent`, `whitenessCorr`, `whittle`, `util.hasBackgroundPool`. None of them is on the Ellmer path (previous section). They fall into three classes: (i) *duplicates* of a live implementation — `Combine`/`BatchIndex` (v1 multi-dataset combination, superseded by `BlockAccumulator`; only caller `test_lsqvce_combine`, 27 points), `varComp`/`vceHutchinson` (Helmert VCE on the flat batch path that `fit` refuses for `Model ="gls"`, `fit.m:577`), `iterateNoiseModel`/`selectArOrder` (an older AR-order loop; `adjustVCE` uses `selectArAcv`); (ii) *instruments* that should run as part of the verdict or the tests but are only reachable by the user typing their name — `snoop`, `systematic`, `datum`, `checkPSD`, `ftest`, `ellipse`, `CommonPrior`, `covComponent`/`toeplitzCofactor` (the $\boldsymbol{V}_n$ of eq. 6.5.16); (iii) *experiments* with no measured benefit on record — `PrefetchLoader`, `poolGroups`, `synthLoader` (a template that G-06 showed drifts from the engine). *Verdict:* class (i) is deleted (no aliases), class (ii) moves into the sweep/verdict or into `dev/tests`, class (iii) is deleted unless a measurement in WP-M0 shows a benefit. About 5 000 lines and 5 test suites leave the tree. *Disposition:* WP-M0, WP-M2.

H-02 Two accumulators, two diagnostics entries, two Helmert implementations. `engine.Accumulator` (713, flat batches, diagonal weights) and `engine.BlockAccumulator` (1284, block-arrow, streaming) implement the same normal-equation/QR accumulation twice; `engine.diagnose` (1016) computes per-observation diagnostics for the flat path while `diagnoseAccum` (235) bridges the streaming path into it; `fit` kinds B/C dispatch to `adjust`/`robust`/`solveNonlinear` on the flat accumulator, kind A to `adjustBlock`/`adjustVCE`/`adjustVCEjoint` on the block one (`fit.m:273-291, 583-626`). A batch problem is a streaming problem with one day, one arc per batch and $n_l = 0$ (Chapter 6); the flat path adds nothing but a second code base to keep in parity. *Verdict:* one accumulator; kinds B/C become adapters that generate a `buildDay` from `A/l/p` or `batchFcn`. *Disposition:* WP-M2.

H-03 Three stochastic-model estimators, one of them an unjustified approximation. `adjustVCE` (2541 lines, the largest file; `run` + `applyShape` + `applyScale` + `applyJoint`) carries mode `separate` (Helmert group scale over `GroupBy` groups *plus* an AR law per stream fitted to the *raw residual autocovariance*, `biasedAcv`/`biasedAcvAxes`, `adjustVCE.m:1809, 2237`) and mode `joint` (pooled PSD by per-lag VCE); `adjustVCEjoint` carries the Ellmer scheme. The residual ACV is a biased estimate of the noise ACV — $\mathrm{E}[\bar{\boldsymbol{v}}\bar{\boldsymbol{v}}^{\mathsf{T}}] = \sigma^2\boldsymbol{R} \neq \sigma^2\boldsymbol{I}$ — with a bias of order $\mathrm{tr}(\bar{\boldsymbol{H}})/m$ per arc, which is small only when $m_a \gg u_{\mathrm{local}}$ and the common-block leverage is small. On GRACE arcs ($m \approx 2000$, $n_l \approx 6$–12) the bias is percent-level at low lags and grows where the local model absorbs low frequencies — exactly the frequencies that decide the KBR weight. The per-lag redundancy s_n that removes this bias is computed by the same tree (`redundancyLags`) for the other two modes. *Verdict (Expert A):* the deviation is not justified in general and is unnecessary, since the exact machinery exists; the AR route is kept as a *whitener choice* (a reduction of the estimated PSD, §2.3), not as an estimator. *Verdict (Expert B):* one estimator with two switches — pooling scope and arc factors — reproduces all three modes.

Disposition: WP-M3 (with a parity test: the new single estimator equals today's `ellmer` output on the WP-51 fixture to 10^{-10}, and equals `pooled` with arc factors off).

H-04 Sweeps are duplicated instead of collected. The residual pass (`engine.residuals`), the redundancy pass (`engine.redundancy`, with its own `mapreduce`), the per-lag pass (`adjustVCE.sweepStreamLags`, `adjustVCE.m:1571`), the whiteness ACV, the contribution probes (`streamContribution`) and the outlier screening (`outliers/snoop`) each re-read and re-whiten the arcs. The v2 design (HANDOVER §3.5, WP-10 "sweep — the ONE second data pass with collectors") was reconciled as done in 2026-08-16 but was never realised in this form. *Verdict:* one sweep with attachable collectors (Chapter 6); PREDICTED saving per VCE pass: 2–4 re-reads of every arc. *Disposition:* WP-M4.

H-05 The verdict tells the user to run engine functions. `report.m:286` prints "Run `lsqvce.engine.systematic()` and fix the functional model"; `report.m:425` "re-snoop — `lsqvce.engine.snoop()` does this for you"; `robust.m:292`, `snoop.m:257/336/351` likewise. The USER tier hands the user an ENGINE name. *Verdict (Expert C):* the systematic-error test and the snooping are collectors of the sweep and methods of `Results` (`R.systematic()`, `R.snoop()`); the verdict names the method. *Disposition:* WP-M4, WP-M5.

H-06 The PSD grid is sized by probing data inside a `try`. `adjustVCE.covGridSize` (`adjustVCE.m:1395-1466`) calls the loader for the first day and arc, catches *any* error and sets `mMax = 0`, then defaults the grid to 128 frequencies. The longest arc is a property of the declared schedule (the resolver) and of the read days (enforced by the accumulator, which sees every arc). *Verdict:* R3 of the coding guide — measured, never probed and defaulted. *Disposition:* WP-M3 (grid = the longest arc seen in pass 1, recorded in the pass record; a user override is a declared choice).

Error handling

H-07 Five empty `catch` blocks and thirteen bare `catch` defaults. Census of the 36 `catch` sites: empty — `adjustVCE.m:1456` (keep a guessed grid), `adjustVCE.m:2050` (`ctx.sd = []`), `BlockAccumulator.m:586` (no local covariance in the log), `util/log.m:196`, `util/showExport.m:63`; bare `catch` → default — `BlockAccumulator.m:577` (`continue`: "system not recoverable"), `adjustVCE.m:1432, 1513, 1566, 2116, 2329, 2332`, `PrefetchLoader.m:184, 198`, `gcvLambda.m:62`, `log.m:123, 218, 225, 306`, `hasBackgroundPool.m:47`, `showExport.m:93`, `mapreduce.m:205` (pool failure becomes an `info.fallback` string). Each converts an undefined variable, a typo or a shape error into a silently different run. *Verdict:* Chapter 7 §1; every site is rewritten as refuse-or-record; the audit counts bare `catch` and targets zero. *Disposition:* WP-M1.

H-08 Output-arity probing by exception, five times. `adjust.m:156`, `robust.m:306`, `solveNonlinear.m:317`, `Combine.m:1013`, `adjustVCE.m:1977` each call the user's function and catch `MATLAB:TooManyOutputs/maxlhs/unassignedOutputs` to learn how many outputs it returns — on *every* call, and a user function that itself raises one of these identifiers is misread. *Verdict:* one wrapper, `nargout(fh)` once at declaration time. *Disposition:* WP-M1.

H-09 369 error identifiers in 40+ unit namespaces, 90 of them in files that H-01 deletes. `lsqvce:combine:` (24), `lsqvce:batchIndex:` (17), `lsqvce:varComp:` (9), `lsqvce:vceHutchinson:` (7), `lsqvce:iterateNoiseModel:` (6), `lsqvce:ftest:` (6), `lsqvce:checkPSD:` (4), `lsqvce:toeplitzCofactor:` (3), . . .; the same rule is raised under different units (`lsqvce:accum:` 28 vs `lsqvce:acc:` 22 for the two accumulators). There is no catalogue and no documented-equals-raised check (the audit's check 15 verifies ids appear in help, not that the catalogue is closed). *Disposition:* WP-M1 (catalogue, Appendix B; target ≈ 130 identifiers).

H-10 Skips are counted in some places and not others. `adjustVCEjoint` counts `skip.cfcnErr/wfcnErr` with the first message (good); `adjustVCE.applyShape` logs

an epoch-covariance failure only at detail verbosity and proceeds on the scalar fallback (`adjustVCE.m:1793`), which is precisely the biased path the L10 rationale warns about; `BlockAccumulator.logLocalSummary` drops a system silently. No skip rate is ever turned into a refusal. *Disposition:* WP-M1, WP-M4 (closed skip enum, threshold).

Interface

H-11 One options object, 57 flat properties, three problems mixed. `Options.m` declares 57 typed properties in ten help groups; the batch-only (`DatumB`, `Sigma0`, `Sparse`, robust, nonlinear), streaming-only (`GroupBy`, `DistributeDays`, Shape/Scale/Outer) and joint-only (`NumLags`, `StepSchedule`, `AvgTail`) knobs sit side by side, and `Model` conflates "iterate or not" with "which estimator" (`gls`/`vce`/`pooled`/`ellmer`). A new user reads 57 rows to find the four that matter. *Verdict (Expert C):* the stock-MATLAB shape is a small top-level object with typed *groups* (`fitlm`'s `RobustOpts`, `optimoptions`' solver-specific sets): ≈ 12 top-level properties, plus `Noise`, `Robust`, `Diagnostics`, `Parallel` groups; `Model` splits into what is estimated (`Estimate`) and how the loop stops. *Disposition:* WP-M5, decision R5-D2.

H-12 The stochastic-model choice lives in two places. What is estimated for a stream is decided by `Options.Model` (global), while the a-priori law lives in `data.systems(k).covariance.(obsType)` (per stream). A stream with a trusted `diag` law and a stream with an unknown red spectrum cannot be treated differently in one run. *Verdict:* the covariance spec gains one field, `estimate = "none" | "scale" | "law"` (+ `arcFactors`, `pool`), so the declaration says per stream what is free; `Options` keeps only run-level policy. *Disposition:* WP-M5, decision R5-D3 (a v5 contract change — additive fields, one breaking rename at most).

H-13 `fit` accepts only an `Options` object; `Results` has no plot; README shows retired names. `fit(data, opts)` requires constructing `lsqvce.Options` first; stock functions take Name-Value directly. `Results` has `report`/`coefficients`/`convergence`/`summary` but no `plot` — the estimated PSD per stream and the pass trajectory are the two pictures every user needs, and both are available in `R.noise`/`R.history` without a design matrix (the WP-57 omission argued about `predict`, which stays omitted). `README.md:60` shows `systemId` and `localsPerArc`, both retired in L10/L11. *Disposition:* WP-M5 (`fit(data, Name=Value)` via a positional `opts` *and* `nv.?lsqvce.Options`; `plot(R, "psd"|"convergence"|"residuals")`), WP-M6 (docs).

Tests

H-14 Category coverage: 3 of 83 units complete, 47 missing three or more categories; metamorphic is a tag; speed budgets are absolute. Audit check 16 (informational, 245 gaps): `report` has contract tests only; `Results` lacks speed and vectorisation; the USER-tier `fit` lacks vectorisation. The WP-29 metamorphic suite exists but is not a counted category. `test_speed` carries absolute budgets and runs locally only. *Verdict:* adopt the seven-category scheme and ratio budgets of `VALIDATION_GUIDE` (Part V, Chapter 16); with H-01/H-02 the unit count drops to ≈ 45, so completeness becomes affordable. *Disposition:* WP-M0 (harness first), WP-M7.

10 Limitations of the method (to document, not to fix)

1. **Stationarity per arc and month.** Non-stationary noise (eclipse transitions, thruster events) is absorbed by arc factors at arc resolution only.
2. **Signal–noise ambiguity.** Unmodelled signal is down-weighted as noise (§3.9); no estimator can tell them apart without a functional model of the signal.
3. **Monte-Carlo redundancy.** $\pm\sqrt{2/Z}$ relative on each trace; no per-observation reliability on the probe route.
4. **Local absorption.** Per-arc empirical parameters remove low-frequency noise power from the estimated law; the law describes the residual process, not the instrument.
5. **Identifiability of nested factors.** Arc and day factors on the same stream are not separable; the contract forbids it, and the theory has nothing better.
6. **Simplified Helmert.** The Förstner iteration ignores the off-diagonal Helmert terms; it converges to the same fixed point but does not deliver the full covariance of the components — only the diagonal Fisher bound is reported.
7. **Scale of u.** Dense $u \times u$ blocks bound the common parameter count to $\sim 2 \cdot 10^4$ on a workstation; beyond that, out-of-core or sparse block strategies are a different toolbox.
8. **Gaps.** Cut, not modelled; the Toeplitz structure is per contiguous arc.

11 The panel's discussion

On collapsing the three noise estimators (H-03)

A: The Ellmer per-lag estimator is the correct one and we already have it; the residual-ACV shortcut was a v1 convenience that survived because it converges to something plausible. Keep the AR reduction as a whitener, not as an estimator.
B: Agreed, with one condition: the "separate" mode is what people run when they have one satellite and 30 days — it must stay *fast*. The per-lag route with probes costs one $u \times Z$ solve per pass and $O(mZ)$ per arc; at $u = 4000$ that is cheaper than the exact leverage the separate mode computes today for its Helmert step. So the unification is a speed-up, not a cost. PREDICTED; WP-M3 measures it.
C: And it deletes `applyShape`/`applyScale`/`applyJoint` and half of `adjustVCE`. The parity test is the risk: today's `vce` mode has no reference other than itself. We pin the new estimator against `ellmer` (which has the Python mirror of `v10review.py`) and accept that `vce`-mode outputs *change* — that is a breaking release and it is the right one.

On the batch kinds (H-02)

C: Kinds B and C exist so that a levelling network or a GNSS batch can use the same verdict. They are a system with one day. Folding them costs one adapter of 60 lines and removes 2 500.
A: The robust IRLS then becomes available on the streaming kind, which it is not today (`lsqvce:fit:notYet`). That is a feature, not a side effect.
B: One caution: the block accumulator's per-arc overhead (arc validation, local-block bookkeeping) must not be paid per row of a 10-row batch. The adapter groups batches into arcs of the user's size; the speed test pins the dispatch ratio.

On the options object (H-11, H-12)

C: 57 flat properties is what happens when every driver's Name-Value list is unioned. The user-facing set is: what to estimate, how many passes, when to stop, how to parallelise, how loud. Everything about a stream belongs with the stream.
B: Per-stream `estimate` also fixes a real workflow: ACC-derived streams have a law we trust from the instrument team; KBR does not. Today one `Model` rules both.
A: And a prior (regularisation) is a stream with `estimate="scale"`. Then there is nothing special about `Reg` either; it is a group.

On how much to simplify

C: Every deletion is a breaking change and the tree has had four breaking releases in a month. Is a fifth right?
B: The four were about the *declaration*, which is now good and stays (R5-D1: the v4 contract is kept, extended additively). v5 is about the *engine* and the options. Users of `fit+data` see a smaller `Options`, a per-stream `estimate` field, and better numbers; they do not rewrite a builder.
A: The one thing that must not be simplified away is the evidence: probe counts, redundancies, skips, condition numbers. Fewer knobs, not fewer numbers.

Part V

The redesign: `lsqvce` v5

12 Design principles and the logic

Six principles, in order of precedence

1. **One estimator, one loop, one accumulator, one sweep.** Every variance component is Ω/r; every iteration is accumulate–solve–sweep–update–gate; every problem kind enters through the same accumulator; every second read is the sweep with collectors.
2. **Declaration says what is free; options say how to run.** What is estimated for a stream is a property of the stream (`estimate`); the pass budget, tolerances, probes, parallel axis and verbosity are properties of the run.
3. **Refuse, report, or proceed.** Chapter 7, mechanically checked.
4. **Measured, never probed.** Every count and grid comes from the declaration or from what was read and enforced; no defaults behind a `try`.
5. **Fewer knobs, not fewer numbers.** Evidence (redundancy, probes, skips, condition, warnings) is always in the result.
6. **Stock-MATLAB ergonomics.** `R = lsqvce.fit(data, Name=Value); plot(R); disp(R)`; typed option groups; identifiers a user can `try/catch`.

12.1 The logic, top down

```
R = lsqvce.fit(data, opts)
  D    = engine.declaration(data)    % validate, derive nc/nl/arcs, one
   Loader per system
  P    = engine.policy(D, opts)      % which components are free, which
   updates run
  noise= engine.noise.init(D)        % per-stream law: PSD grid, whitener,
   arc factors = 1
  acc  = engine.Accumulator(D.nc, opts.Backend)
  for pass = 1:opts.MaxPasses
      acc.reset();
      engine.accumulate(acc, D, noise, opts.Parallel)   % parallel over days
   , fold in day order
      sol  = acc.solve();                                % factor, condition
   , NaN-honest sigma0
      S    = engine.sweep(acc, D, noise, sol, P.collectors, opts.Parallel)
                                                         % residuals, (
   Omega_n, s_n), r_i, whiteness, worst-K
      [noise, x0, upd] = engine.update(P, noise, S, sol) % VCE / IRLS /
   Gauss-Newton, in that order
      hist(pass) = engine.passRecord(pass, sol, S, upd) % skips, norms,
   whiteness, timing, gate
      if engine.gate(hist, opts), break, end
  end
  R = lsqvce.Results(sol, S, hist, noise, D, engine.runMeta(opts))   %
   verdict on demand: R.report
```

`Model="gls"` is `MaxPasses = 1` with an empty policy; every other model is a policy. The batch kinds are `engine.declaration` recognising `A/l/p` or `batchFcn` and synthesising one system with a generated `buildDay`.

13 The user interface

13.1 Three lines, all kinds

```
R = lsqvce.fit(data);                                          % GLS under the
   declared laws
R = lsqvce.fit(data, Estimate="noise", MaxPasses=8);     % VCE: laws, arc
   factors, group scales as declared
R = lsqvce.fit(data, Estimate="noise", Robust="huber");  % plus IRLS on the
   same loop
R = lsqvce.fit(struct(A=A, l=l, p=p), Robust="igg3");    % an in-memory
   problem, same verdict
R.report;  T = R.coefficients;  plot(R, "psd");  plot(R, "convergence")
```

13.2 `lsqvce.fit`

```
function R = fit(data, opts, nv)
arguments
    data (1,1) struct
    opts (1,1) lsqvce.Options = lsqvce.Options()
    nv.?lsqvce.Options                          % Name=Value overrides, stock-
   MATLAB style
end
```

The positional `opts` and the Name-Value overrides are *one* mechanism (the overrides are applied to `opts` and validated by the class); the recorded `R.meta.options` is the resolved object. Errors of the declaration carry `lsqvce:fit:<rule>`; nothing is read before validation completes.

13.3 `lsqvce.Options` — twelve properties and four groups

property	type / default	meaning
`Estimate`	`"none"` | `"noise"`	`"none"`: one GLS pass under the declared laws. `"noise"`: run the VCE policy on every stream whose spec declares something free (`estimate = "none"`). Replaces `Model`; the pooled/ellmer distinction moves to the stream spec.
`Robust`	`"none"` | `"huber"` | `"igg3"` | `"bisquare"`	IRLS policy on the same loop; details in `Robust` group
`MaxPasses`	8	outer pass budget (1 for `Estimate="none"`)
`Tolerance`	5e-3	stop when every free component moved less than this (relative)
`Whiteness`	0.01	Ljung–Box level; a stream is "white" above it; stop when all are
`Backend`	`"normal"` | `"qr"`	§1.3
`Probes`	100	Hutchinson probes for the per-lag redundancies; `0` = exact route

`Seed`	[]	RNG seed for the probes; recorded
`Parallel`	group	`Mode "auto"|"serial"|"pool"`, `Axis "days"|"arcs"`, `Threshold 8`
`Noise`	group	`Grid` (`"auto"` = longest arc), `Whitener "toeplitz"|"ar"`, `MaxOrder 60`, `Floor 1e-3`, `Smooth 0`, `MeanType "robust"|"plain"`, `FreezeRedundancy 1`, `SkipThreshold 0.2`
`Robust`	group	`Tuning []` (= influence default), `MaxIter 30`, `Tol 1e-8`
`Diagnostics`	group	`Alpha 1e-3`, `Bonferroni true`, `Beta 0.2`, `RWeak 0.1`, `Test "auto"|"pope"|"baarda"`, `Store "flagged"|"all"|"none"`, `NWorst 1000`, `Systematic true`
`Verbose`	1	0..3
`LogFile`	""	path; "" = none

Regularisation is not an option: a prior is a stream (§13.4). Nonlinear is not an option: `data.x0` declares it. `DistributeDays`, `ParallelThreshold`, `GroupBy`, `VceMode`, `NumLags`, `Norm`, `StepSchedule`, `AvgTail`, `C0`, `PsdMode`, `ComponentsPerEpoch`, `DatumB`, `Sigma0`, `Sparse`, `Reg*`, `ShowSeconds`, `ExportDir`, `LogFid`, `LogFlag` and the `Vce*`/`Joint*`/`Nonlinear*`/`Outer*` names leave; Appendix A maps each.

13.4 The data contract, v5 delta

The v4 struct stays (R5-D1). Additive changes only:

- `covariance.(obsType).estimate` ∈ {`"none"`, `"scale"`, `"law"`}, default `"none"`. `"scale"` = one variance factor for the stream (Helmert on a fixed law); `"law"` = the per-lag PSD estimate (Ellmer).
- `covariance.(obsType).arcFactors` logical, default `true` when `estimate="law"`; `pool` ∈ {`"stream"`, `"obsType"`}: whether the law is shared with the same observation type of other systems (today's "pooled").
- `covariance.(obsType).dt` required for time-domain laws (`acf`, `arma` given in seconds); a sample-domain law states `dt = []` explicitly.
- `data.priors(k)`: `struct(L=, Q=, sigma=, estimate=)` — regularisation as declared pseudo-observation groups with their own component.
- Kinds B/C unchanged; `Robust` becomes available on kind A.

One breaking rename is proposed for decision (R5-D3): `covariance` → `noise`, because the field now declares more than a covariance; the panel is split (A, B for; C against on churn grounds) and leaves it to the user.

13.5 `lsqvce.Results`

Immutable: `x`, `Cxx`, `sigma0`, `converged`, `stopReason`, `history`, `noise` (per stream: law, PSD, arc factors, whiteness, redundancy of every component), `skips`, `systems`, `meta`. Methods: `report`, `coefficients(Alpha=)`, `residuals(Stream=, Kind="whitened"|"raw")` (from the stored worst-K or a re-sweep on request), `redundancy`, `reliability`, `systematic`, `snoop(MaxReject=)`, `contribution`, `convergence`, `summary`, `plot(kind)`, `disp`. Everything the verdict tells the user to do is a method on `R`.

14 The function set

≈ 46 public names (from 83): USER 4, ENGINE 22, KERNEL 15, UTIL 5. "Replaces" names the v4 file(s) whose responsibility the function absorbs.

name	responsibility (signature)	replaces
USER		
`fit`	`R = fit(data, opts, Name=Value)`	fit
`Options`	typed value class, 12 properties + 4 groups	Options
`Results`	handle class, methods above	Results
`report`	`V = report(R, Level=, ...)` verdict printer	report
ENGINE — declaration and data		
`declaration`	`D = declaration(data)`: validate all rules, derive nc/nl/arcs/obsTypes, scope plans, one `Loader` per system, batch-kind adapters	fit (validation half), makeLoader (plans), resolveNc, buildScopePlan, adjust/robust/solveNonlinear (kind adapters)
`Loader`	class: `day(d, obsType) -> arcs`; caching; builder wrapper (nargin/nargout once; cause-wrapped errors); arc resolver; epoch-covariance hook	makeLoader, applyCovOverride, PrefetchLoader, synthLoader
`scopePlan`	`plan = scopePlan(tables, calendar)`: expansion of scoped templates, offsets, `info` for `buildDay`	buildScopePlan, localsPerArc plan
ENGINE — linear algebra		
`Accumulator`	class: backend, common block, per-system scope stores, `addArc`, `closeScope`, `solve`, `backSubstitute`, `condition`, `setPrior`	BlockAccumulator, Accumulator, Combine, BatchIndex, ReducedContribution, CommonPrior, applyRegularisation, datum
`NormalEqBackend`, `QrBackend`	the two factor stores; `add`, `solve`, `sigma0` (NaN-honest), `pivots`	NormalEqSystem, QRSystem
`accumulate`	`accumulate(acc, D, noise, par)`: parallel over the chosen axis, deterministic fold	BlockAccumulator .processSystem / .processDay, mapreduce
`arcWhiten`	`[Ab, Bb, lb, W] = arcWhiten(arc, law)`: Toeplitz / AR / VAR / separable dispatch, overlap batching	covModel, arcBatchWhiten
ENGINE — the loop		
`iterate`	the loop of Chapter 12; owns the pass record and the gate	adjustVCE.run, adjustVCEjoint, adjustBlock, iterateNoiseModel
`policy`	`P = policy(D, opts)`: which components are free, which collectors attach, which updates run	fit dispatch, jointModelOpt, resolveSatSpecs

`sweep`	`S = sweep(acc, D, noise, sol, collectors, par)`: ONE second pass	residuals, redundancy, collectResiduals, streamContribution, adjustVCE.sweepStreamLags, outliers, leverage, leverageBlocks, localDiagnostics
`collectors`	package: `residualNorms`, `lagStatistics`, `observationRedundancy`, `whiteness`, `worstK`, `contribution`, `systematic`	(the collectors' code, gathered)
`probeBlock`	`Z = probeBlock(U, Probes, Seed)`: $\boldsymbol{U}^{-1}\bar{\boldsymbol{Z}}$ once per pass	sharedProbe, probeMatrix, probeStream, traceEstimate
`update`	`[noise, x0, upd] = update(P, noise, S, sol)`: dispatches the three policies	psdVCE, arcVarianceFactors, vce, varComp, vceHutchinson, robust (IRLS core), solveNonlinear (GN core)
`noiseLaw`	class: PSD/ACV/AR/VAR representation with `dt`, grid, floor, taper, `whitener()`, `arcFactor(a)`, `redundancy(component)`	covModel (model half), jointModelOpt, checkPSD (as a method)
`gate`	`[stop, reason] = gate(hist, opts)`	adjustVCE convergence logic
`passRecord`, `runMeta`	provenance	runMeta
ENGINE — judgement		
`verdict`	`V = verdict(R, opts)`: levels 0–4 from the sweep's collectors (no re-read)	report (computation half), diagnose, diagnoseAccum, stat (statistics half), systematic, snoop, ftest, ellipse
KERNEL		
`dctI`, `psd2acv`, `acv2psd`	transforms, one normalisation	dctI, psd2acv
`levinsonDurbin`, `ar2arma`, `armaACV`, `armaWhiten`	AR/ARMA	unchanged
`toeplitzWhiten`	Levinson/Schur Toeplitz Cholesky, $O(m^2)$	decorrFactor, toeplitzCofactor, covComponent
`whittle`, `varACV`, `varWhiten`, `separableWhiten`	multi-axis	(whittle promoted from engine)
`redundancyLags`	per-arc (Ω_n, s_n), exact or probe	redundancyLags
`residualAcv`, `ljungBox`	whiteness statistics	residualAcv, whitenessCorr, crossCorr, adjustVCE.ljungBoxWhite
`stat`	χ^2, τ, normal quantiles, MAD; base MATLAB	stat
UTIL		

`log`, `verbosity`, `mergeStruct`, `getfielddef`, `callUser`	logging; the one user-function wrapper	log, verbosity, mergeStruct, getfielddef, showExport (deleted), hasBackgroundPool (deleted)

15 Workflow, data handling, parallelism

The workflow is §5.2; what changes relative to v4 is where the work happens:

step	v4 (who)	v5 (who)
validate	`fit` (500 lines) + `makeLoader` (relabelled ids)	`engine.declaration`, one identifier table
count nc/nl/arcs	`fit`, `makeLoader`, `resolveNc`, `buildScopePlan`	`scopePlan`
whiten	`covModel` per stream per pass, rebuilt from overrides	`noiseLaw.whitener()` cached until the law changes
accumulate	`BlockAccumulator.processSystem` via `mapreduce`	`accumulate` over `Parallel.Axis`, fold in order
residual passes	3–5 separate re-reads per pass (H-04)	one `sweep`
update	three drivers	`update` with the policy list
judge	`report` + user-run engine functions	`verdict` from the sweep; `R` methods

Parallel plan. `Parallel.Axis="days"` is the default: one worker builds, whitens, eliminates and returns the raw day partial (common block, $u \times u$ dense) — the transfer is $8u^2$ bytes per day, 128 MB at $u = 4000$; for $u > 6000$ the plan switches to `Axis="arcs"` with worker-side folding across the arcs of a day and a client fold per day (PREDICTED crossover; WP-M4 measures). The sweep parallelises over the same axis; every collector's fold is associative and applied in day order; the probe block is broadcast once per pass (a `parallel.pool.Constant`). Nothing about pool type is decided by the toolbox; `Threshold` is the number of tasks below which the pool overhead loses, measured by the speed suite on the user's machine and stored in the pass record.

Memory plan. Two $u \times u$ blocks (accumulated, factor) + $u \times Z$ + per-worker one day; `Store="flagged"` keeps only flagged residuals by default (v4 keeps `"all"`, which at GRACE scale is 10^6 rows per stream per pass).

16 Tests

16.1 Categories (seven, counted per unit) and tags

Adopted from `VALIDATION_GUIDE` Part 1: `contract`, `reference`, `precision`, `speed`, `robustness`, `vectorisation`, `metamorphic`; tags `diagnostics`, `provenance`, `layering`, `concurrency`, `convergence`. Audit check 16 counts seven categories; an exemption is one row per (unit, category) with a reason; `contract` is never exempt for a validating unit. Speed budgets are ratios with expected, provenance and band, timed by one helper (`dev/tests/private/timePair`) before any budget is written (WP-M0).

16.2 The oracle register

id	oracle	certifies	status
O1	dense closed form (small n, u): $\boldsymbol{N}^{-1}$, $\boldsymbol{H}$, r_i, Helmert matrix, per-lag s_n by explicit tr($\boldsymbol{RV}_n$)	every estimator and every redundancy at machine precision	exists (test_vceMath, test_precision); extend to s_n
O2	`validation/v10review.py` (Python mirror of the Ellmer scheme, 10^{-14})	the per-lag VCE, DCT pairing, arc factors	exists; re-run against v5 `update`
O3	WP-48 physical oracle (shDesign, planted truth, z-scores)	recovery of a gravity field under coloured noise	exists
O4	GROOPS normal-equation file of one arc/day (user-supplied)	whitening + accumulation parity with the production system	OPEN — the only external authority; WP-M3 may not close without it or an explicit user waiver
O5	Koch (1999) worked VCE example; Förstner (1979) redundancy example	group Helmert, reliability numbers	to add (cheap)
O6	analytic invariants: $\sum r_i = n - u$; $\Omega_g = r_g$ at the fixed point; DCT self-inverse; Toeplitz PD $\Leftrightarrow$ PSD > 0; arc factors unit mean	every pass, every run	exists partly (level 0); make them collectors' self-checks
O7	`gracefoReading` real L1B day (RL04, 2020-01-01, PO.DAAC)	the builder example end to end; gaps as arc boundaries	to add with WP-M6

16.3 Per-unit matrix (what each category asserts)

unit	category	assertion

`fit`	contract	every declaration identifier fires alone; unknown field names the field; Name-Value overrides equal the object route byte-identically
	reference	`fit == iterate` on every kind (dispatch adds nothing)
	metamorphic	system order, day order, observation-type order invariance (eps-level: fold order); duplicating a system halves its component (bitwise on the estimate up to the fold); scaling $\boldsymbol{l}$ by c scales $\hat{\boldsymbol{x}}$ by c and $\hat{\sigma}_0$ by c (bitwise)
	speed	`fit`/`iterate` ratio ≤ 1.05 at $N = 3$ systems $\times$ 2 days
	robustness	builder throws (cause chain), builder returns wrong shapes, empty day, pool absent/present, `nl = 0`, single arc, single observation type
`Accumulator`	precision	NEQ vs QR to $\kappa\varepsilon$; back-substitution reproduces the joint solve; Schur elimination equals the joint system (O1)
	robustness	rank deficiency refused with d; datum by prior; cancellation NaN both backends
	concurrency	bit-identical across 1/2/N workers (fold order)
`sweep`	reference	every collector against O1 dense forms
	vectorisation	batched arcs equal per-arc calls bitwise
	metamorphic	collectors are order-invariant under day permutation (eps); adding a collector does not change another's output (bitwise)
`update`	reference	O2 Ellmer mirror 10^{-12}; Förstner iteration reaches $\Omega_g = r_g$ (O6)
	convergence	planted PSD recovered as z-scores against the Fisher bound; pass count to tolerance monotone in M
	robustness	frozen component below redundancy; non-PD law after update skipped and counted; alternating scale detected
`noiseLaw`	precision	PSD→ACV→PSD round trip; AR reduction reproduces ACV within stated tolerance (checkPSD as a method)
	metamorphic	resampling by Δt and lag index agree; taper preserves PD
`verdict`	reference	O5 worked examples; O6 level-0 identities
	contract	every skip reason reaches the report; warning inventory once per run
`Results`	vectorisation	`coefficients` on 1 and 10^4 parameters; `plot` on 1 and 10 streams
`Loader`	contract	every builder identifier; nargin 2/5/6/varargin; ragged ids as gaps

16.4 Gates

Green = zero failures $\wedge$ every suite loaded $\wedge$ declared = executed per suite $\wedge$ no new warning identifier $\wedge$ no ratio budget exceeded $\wedge$ manifest verified $\wedge$ audit exit 0 $\wedge$ bare-`catch` count 0 $\wedge$ identifier catalogue closed. Predicted point count stated before every run and reconciled three ways (VALIDATION_GUIDE Part 7). Postflight after every merge: `ci_gate` on R2026a incl. `test_parallel`, and every example smoke-run.

17 Documentation

One source per fact: `DATA_CONTRACT.md` (declaration, v5 delta) → `fit` help and the getting-started page are generated from it, not copied; the identifier catalogue → ERRORS blocks checked against it; `Options` help → the class's own property metadata. Deliverables: tiered HTML (`doc lsqvce`: Getting started; USER reference; ENGINE; KERNEL), `lsqvce_theory.pdf` = Parts I–III of this document (LaTeX source under `dev/docs_src/theory_r5`, built by `build.sh`; supersedes the ReportLab pipeline for the theory PDF, R5-D6), the manual reduced to the user path (chapter 0 + D + examples), `RELEASE_NOTES` with the 83→46 mapping (Appendix A), and one new example: a real GRACE-FO day from `gracefoReading` into a `buildDay` (O7). Every help text keeps the INPUT/OUTPUT/ERRORS/ACCURACY/EXAMPLE blocks and the dated model stamp.

18 Work packages, levels, decisions

18.1 Levels M0 . . . M7

One level = one branch = one PR (HANDOVER §11.5). Order of work is M0 → M1 → M2 → M3 → M4 → M5 → M6 → M7; M3 and M4 may be swapped if the GROOPS oracle (O4) is late. Nothing below starts before the decisions of §18.2 are taken.

level	WP	scope	exit criterion
M0	78	**Instruments first.** Timing helper (rotation, 15 repeats, per-repeat statistic), seven-category audit (check 16 → 7 categories, metamorphic counted), bare-`catch` audit metric, identifier-catalogue check (documented = raised = catalogued), layering check on `tiers.csv`; fault-injection self-test for each new check	each check fires on a broken fixture and is silent on `main`; suite green; no functional change
M0	79	**Measurements that decide deletions.** PrefetchLoader vs plain parfor over days; exact vs probe sweep at $u \in \{500, 2000, 4000\}$; per-arc overhead of the block accumulator on 10-row arcs (H-02 caution)	numbers in `dev/DEVELOPMENT.md`; deletions in M2 cite them
M1	80	**Error doctrine** (Chapter 7): rewrite all 36 `catch` sites; one `util.callUser` wrapper (nargin/nargout once, cause-wrapped); closed skip enum + counters + `SkipThreshold`; warnings once per run + inventory in `meta`; `stopReason`; identifier catalogue (Appendix B) with the dead namespaces removed	bare-`catch` count 0; every identifier fires in a contract test; skip counts reach `report`; RELEASE_NOTES lists retired identifiers
M2	81	**One accumulator.** `engine.Accumulator` = BlockAccumulator + scope stores + prior; delete `Accumulator` (flat), `Combine`, `BatchIndex`, `ReducedContribution`, `CommonPrior`, `applyRegularisation`, `datum` (absorbed), `adjust`, `robust`, `solveNonlinear` (cores move to `update`); kinds B/C via `declaration` adapters; `data.priors`	kinds B/C outputs byte-identical to v4 on the test fixtures (reference); IRLS on kind A available; 5 suites retired, points re-homed
M2	82	**Delete the unreached duplicates** (H-01 class i and iii, per WP-79 numbers): varComp, vceHutchinson, iterateNoiseModel, selectArOrder, PrefetchLoader, synthLoader, poolGroups, resolveSatSpecs, normalizeSatList, showExport, hasBackgroundPool; instruments (class ii) move to `dev/tests` or into the verdict (M4)	audit: no removed name referenced anywhere (check 2); glob-inventoried, not hand-picked

M3	83	**One noise estimator.** `engine.noiseLaw` (PSD-domain representation with `dt`, grid from the schedule, floor, taper, whitener cache); `engine.update` VCE policy = per-lag $(\Omega_n, s_n) \to$ PSD amplitudes $\to$ arc factors $\to$ group scales; `estimate`/`arcFactors`/`pool` on the stream spec; AR/ARMA as whitener choice; delete `applyShape`/`applyScale`/`applyJoint`, `covModel`, `jointModelOpt`, `psdVCE`, `arcVarianceFactors`, `vce`, `checkPSD` (method now)	parity: v5 `estimate="law"` equals v4 `ellmer` on the WP-51 fixture to 10^{-10} (O2), equals v4 `pooled` with `arcFactors=false`; v4 `vce` mode documented as changed (breaking) with the z-score recovery on O3 not worse; O4 GROOPS parity or waiver
M3	84	**Speed of the unified path** (H-03 B's condition): budgets as ratios against v4 `ellmer` and v4 `vce` at the WP-79 points	per-pass ratio ≤ 1.0 vs v4 `ellmer`; ≤ 1.5 vs v4 `vce` (PREDICTED; measured decides)
M4	85	**One sweep with collectors +** `engine.iterate`/`policy`/`gate`/`passRecord`; `accumulate` over `Parallel.Axis`; delete `residuals`, `redundancy`, `collectResiduals`, `streamContribution`, `contribution`, `outliers`, `leverage`, `leverageBlocks`, `localDiagnostics`, `mapreduce`, `sharedProbe`/`probeStream` (into `probeBlock`), `adjustVCE`, `adjustVCEjoint`, `adjustBlock`	one re-read per pass (counted by a loader instrument); collectors against O1; concurrency bit-identical; `Axis="arcs"` measured at $u = 4000$
M4	86	**Verdict from the sweep**: `engine.verdict` absorbs diagnose/diagnoseAccum/stat(tests)/systematic/snoop/ftest/ellipse; `Results` methods `systematic`, `snoop`, `reliability`, `plot`	report never names an engine function; O5/O6; every level-2/3 number traceable to a collector
M5	87	**Interface**: `Options` v5 (12 + 4 groups), `fit(data, opts, Name=Value)`, contract v5 delta (+ rename if R5-D3), `Results` v5 fields, `plot`/`disp`; version 5.0; RELEASE_NOTES 83→46	3-line acceptance on every kind; Name-Value == object byte-identical; every retired option name in the migration table
M6	88	**Documentation**: theory PDF from this LaTeX source (R5-D6), tiered HTML regenerated from the new tiers, DATA_CONTRACT v5, manual reduced, README onto v5 (fix the v2 names), `gracefoReading` example (O7), help sweep with ACCURACY blocks	audit check 17 doc sync; README examples run; theory PDF builds in CI (pdflatex on the runner or committed PDF with a source hash check)
M7	89	**Seven-category completeness**: every unit of the 46 has all seven or an exemption row; ratio speed suite; metamorphic suite counted; fault-injection for gates; `test_parallel` on a PCT-licensed runner (WP-30 backlog) or documented local-only	check 16 gaps 0; exemptions ≤ 20 rows, each argued

Backlog carried: WP-42 (monotone/trust-region alternation — revisit after M3: with one scale owner it may be moot), WP-31 GPU (gated on M4 measurements), WP-30 runner.

18.2 Decisions required

#	decision	recommendation
R5-D1	Keep the v4 data contract as the base of v5 (additive `estimate`/`arcFactors`/`pool`/`dt`/`priors`); no rewrite of the declaration	**keep** (all three experts)
R5-D2	Replace `Model` by `Estimate="none"|"noise"` + per-stream `estimate`; `Options` reduced to 12 + 4 groups (Chapter 13)	**yes**; breaking, in M5 with the migration table
R5-D3	Rename `covariance` → `noise` in the stream spec	split panel; C recommends **no** (churn on every builder demo and fixture); A/B mildly for. Default if silent: no
R5-D4	Retire the biased-ACV "separate" estimator (v4 `vce`) in favour of the per-lag estimator; accept changed numbers in that mode	**yes** (H-03); the z-score recovery test on O3 is the acceptance, not equality to v4
R5-D5	Delete H-01 classes (i) and (iii) outright (no ENGINE-tier survival)	**yes** under the no-aliases rule; instruments (class ii) survive as `dev/tests` tools or verdict collectors
R5-D6	Theory authority: this document's Parts I–III replace `lsqvce_theory.pdf` (LaTeX source, pdflatex build); ReportLab theory pipeline retired	**yes**; the manual keeps its own pipeline until M6 decides
R5-D7	Sequence: M0/M1 first (instruments and error doctrine on the *current* tree) so every later deletion is measured and every catch is already honest	**yes**; M0 has no functional change and can merge within a day

18.3 Risks and how the plan bounds them

- **Parity of the unified estimator** (M3). Bounded by O2 (Python mirror, exists), O3 (physical oracle, exists) and O4 (GROOPS, open). If O4 stays open, M3 merges on O2+O3 with the waiver recorded in the oracle register — an empty row, not a silence.
- **Speed regression of the block accumulator on tiny batches** (M2). Measured in WP-79 before the deletion; the adapter's arc size is the lever.
- **Hand-picked inventories** (the lesson pinned twice in §11.8). Every deletion and rename list in M2/M4/M5 is produced by `audit_tree.py` globs and committed with the WP.
- **Breaking churn.** Exactly one breaking release (5.0) at M5; M0–M4 are internal and non-breaking for a `fit+data` user except the numbers of the retired `vce` mode (R5-D4), which are announced in the M3 PR body.
- **The theory PDF build in CI.** pdflatex on the hosted runner adds a job; alternative is the committed PDF with a source-hash check (audit). M6 decides by measurement of the runner image.

18.4 What does not change

The four USER names; the `data` struct and `buildDay`; NaN honesty; NEQ default/QR fallback; the probe contract and seeds; the five-level verdict; the level-based loop, the audit gate, the user as sole merger; the dated model stamp in every file; the HANDOVER as the single status document.

A Old → new mapping (83 → 46 public names)

Generated from `dev/docs_src/tiers.csv` at `d7ad051`; "→" = absorbed by, "deleted" = no successor (H-01 class i/iii), "dev" = moves to `dev/tests` as an instrument.

v4 name	tier	v5
fit, Options, Results, report	USER	unchanged names, new signatures (M5)
engine.Accumulator	ENGINE	→ engine.Accumulator (the block one; flat path deleted)
engine.BlockAccumulator	ENGINE	→ engine.Accumulator
engine.BatchIndex, engine.Combine	ENGINE	deleted (M2)
engine.CommonPrior, engine.applyRegularisation, engine.datum, engine.ReducedContribution	ENGINE	→ engine.Accumulator (priors, rank evidence, scope stores)
engine.NormalEqSystem, engine.QRSystem	ENGINE	→ engine.NormalEqBackend, engine.QrBackend
engine.PrefetchLoader, engine.synthLoader	ENGINE	deleted (M2; template moves to `template/`)
engine.adjust, engine.robust, engine.solveNonlinear	ENGINE	→ engine.declaration (adapters) + engine.update (IRLS, GN cores)
engine.adjustBlock, engine.adjustVCE, engine.adjustVCEjoint	ENGINE	→ engine.iterate + engine.policy + engine.update (M4)
engine.arcVarianceFactors, engine.psdVCE, engine.vce, engine.varComp, engine.vceHutchinson	ENGINE	→ engine.update (VCE policy) (M3); varComp/vceHutchinson deleted (M2)
engine.checkPSD	ENGINE	→ engine.noiseLaw.check (method)
engine.collectResiduals, engine.residuals, engine.redundancy, engine.leverage, engine.leverageBlocks, engine.localDiagnostics, engine.outliers, engine.streamContribution, engine.contribution	ENGINE	→ engine.sweep + engine.collectors.* (M4)
engine.covComponent	ENGINE	→ kernel.toeplitzWhiten (private helper) / dev
engine.covModel, engine.jointModelOpt	ENGINE	→ engine.noiseLaw + engine.arcWhiten (M3)
engine.diagnose, engine.diagnoseAccum, engine.stat (tests), engine.systematic, engine.snoop, engine.ftest, engine.ellipse	ENGINE	→ engine.verdict + Results methods (M4); kernel.stat keeps the distributions
engine.gcvLambda	ENGINE	dev (instrument; VCE is the default weight)
engine.iterateNoiseModel, engine.selectArOrder	ENGINE	deleted (M2)
engine.selectArAcv	ENGINE	→ engine.noiseLaw.reduceAR (method)
engine.makeLoader, engine.resolveNc, engine.satKeyOf	ENGINE	→ engine.declaration, engine.Loader, engine.scopePlan
engine.mapreduce	ENGINE	→ engine.accumulate / engine.sweep (the fold is theirs)
engine.normalizeSatList, engine.resolveSatSpecs, engine.poolGroups	ENGINE	deleted (M2)
engine.runMeta	ENGINE	unchanged (+ passRecord)
engine.sharedProbe, engine.traceEstimate	ENGINE	→ engine.probeBlock

engine.whittle	ENGINE	→ kernel.whittle
kernel.ar2arma, armaACV, armaWhiten, dctI, levinsonDurbin, psd2acv, redundancyLags, residualAcv, separableWhiten, varACV, varWhiten	KERNEL	unchanged (+ acv2psd, ljungBox, toeplitzWhiten)
kernel.decorrFactor, kernel.toeplitzCofactor	KERNEL	→ kernel.toeplitzWhiten
kernel.crossCorr, kernel.whitenessCorr	KERNEL	→ kernel.ljungBox / engine.collectors.whiteness
util.log, util.verbosity, util.mergeStruct, util.getfielddef	UTIL	unchanged (+ util.callUser)
util.showExport, util.hasBackgroundPool	UTIL	deleted (M2)

B Error identifier catalogue (v5 authority, initial)

Grammar `lsqvce:<unit>:<reason>`. The catalogue is a CSV in `dev/docs_src/error_ids.csv` (unit, reason, class, message template, since); the audit refuses an identifier raised or documented but absent here. Initial content, by unit (reasons abbreviated; ≈ 130 in total after M1):

unit	reasons
fit	unknownField, missingField, noSystems, noCommon, noDays, badDays, badId, duplicateSystem, badScope, badArcs, builderKind, paramType, kindAmbiguous, modelKind (retired in M2), notYet (retired in M2)
Options	badValue (typed validators carry MATLAB ids), badCombination, badGroup, badNumLags (retired)
declaration	(the fit rules above move here; fit re-raises with its unit for the user)
Loader	builderThrew (cause), badDay, dayShape, arcBadNc, arcBadNl, badArc, badArcId, nonDeterministic (two reads differ), hookThrew
scopePlan	badScope, duplicateTemplate, emptyExpansion
Accumulator	rankDeficient, cancellation (sigma0 NaN), badPrior, scopeNotClosed, backendMismatch, badArcSize
NormalEqBackend, QrBackend	notPD, cancellation, rankDeficient
noiseLaw	nonPD, badSigma, unstableAR, lawTooShort, dtMissing, badGrid, floorHit (warning), tapered (warning)
arcWhiten	nonPD (skip), batchOverlap
update	frozen (warning), alternating (warning), negativeComponent (warning: clamped), skipThreshold (refusal)
iterate	maxPasses (warning), notConverged (warning), poolFallback (warning)
sweep	collectorFailed (cause), skipped (warning, per reason count)
verdict	notComputed (reliability on probe route), integrity (level-0 violation: refusal)
Results	noHistory, noCovariance, noSweep, badKind (plot)
report	badLevel
kernel.*	one `badInput` per function plus the specific ones (levinson: notPD; ar2arma: unstable; dctI: badNorm; redundancyLags: badProbes)
util.callUser	nargin, nargout, threw (cause)

C Options mapping v4 → v5 (as implemented, M5)

As of 5.0 — a dated record

This table is version 5.0 as shipped on 3 September 2026. The option set has moved since (5.1 … 6.1: `MinPasses` and `LogFile` retired, `Robust` composable with `Estimate="noise"`, 17 knobs); the current table is the Manual, chapter 4 (R6 WP-106).

The plan's table (Part V, "The user interface") was the intent; this is what version 5.0 ships (`RELEASE_NOTES.md` and `dev/HANDOVER.md` §12.10 are the same table). 14 top-level knobs, five typed groups.

v4	v5
Model gls	Estimate="none" (default)
Model ellmer	Estimate="noise" (Pool as wished, ArcFactors=true default)
Model vce	Estimate="noise", Pool="stream", ArcFactors=false (changed numbers, R5-D4: one estimator; acceptance by the physical oracle)
Model pooled	Estimate="noise", Pool="all", ArcFactors=false
Robust="irls" + Influence	Robust="huber" | "igg3" | "bisquare"
Tuning, RobustMaxIter, RobustTol	RobustOpts.Tuning / .MaxIter / .Tol
MaxOuter, JointMaxIter, VceMaxIter, NonlinearMaxIter	MaxPasses (one budget); MinPasses new
JointTol, OuterTol, VceTol, TolX	Tolerance
TolF, Lambda0	GaussNewton.TolF / .Lambda0
NumLags	Noise.Grid
PsdMode	Noise.Whitener ("toeplitz" default, as before)
MeanType, Smooth, MinRedundancy, SkipThreshold	Noise.MeanType / .Smooth / .FreezeRedundancy / .SkipThreshold
Parallel, DistributeDays, ParallelThreshold	Parallel.Mode / .DistributeDays / .Threshold (Parallel.Axis: gated on the WP-79 measurement, M7)
Alpha, Bonferroni, Beta, RMin, RWeak, Test, RobustCheck, Store, NWorst	Diagnostics.<same>
Verify, Progress	always on / retired (Verbose)
LogFid, LogFlag	LogFile
ShowSeconds, ExportDir, plotPostSolve, plotPostModel	OnPass (the per-pass inspection hook: pass state, live accumulator, the declaration, processingOptions; return false to stop) and R.plot(kind) post mortem
DatumB	data.priors = struct('L', B', 'sigma', 1) – the exact minimum-constraint solution
Sigma0, Sparse	retired (the a-posteriori σ_0 is the report; sparse normals measured moot)
Reg, RegLambda, RegMatrix	unchanged (block-path regularisation); a prior with its own component is data.priors (its VCE component: M7)
GroupBy, ComponentsPerEpoch, MaxOrder, AcvTol, MaxLag, WhiteAlpha	retired with the separate mode (groups are streams; the axis count comes from the builder; whiteness fixed at $K = 20$, $\alpha = 0.05$)
Norm, C0, StepSchedule, AvgTail	engine tier only (adjustVCEjoint)
Backend, Probes, Seed, Verbose	unchanged; CacheDays, Pool, ArcFactors, OnPass new

Deviations from the plan, and why: `noiseLaw` was not created (the PSD spec, covModel's resampling and two policy maps are the mechanism; a class would add a name); `iterate`/`policy`/`gate`/`passRecord` are the joint driver's loop, fit's policy mapping, `stopReason` and `history`; `Noise.Sigma0Prior` was not needed (the whitened frame's a-priori σ_0 is 1 by construction); `Parallel.Axis` waits for its measurement; the per-pass plot hook was first retired and then restored as `OnPass` because the project's opening brief required it – the rule recorded in HANDOVER §12.9 (5c′): a review-plan item never outranks an opening-brief requirement.

D References

- Baarda, W. (1968). *A testing procedure for use in geodetic networks*. Netherlands Geodetic Commission, Publ. on Geodesy 2(5).
- Bekas, C., Kokiopoulou, E., Saad, Y. (2007). An estimator for the diagonal of a matrix. *Appl. Numer. Math.* 57, 1214–1229.
- Ellmer, M. (2018). *Contributions to GRACE gravity field recovery: improvements in dynamic orbit integration, stochastic modelling of the antenna offset correction, and co-estimation of satellite orientations*. Doctoral thesis, Graz University of Technology. Ch. 2, §6.5.
- Förstner, W. (1979). Ein Verfahren zur Schätzung von Varianz- und Kovarianzkomponenten. *Allg. Vermessungs-Nachr.* 86, 446–453.
- Golub, G.H., Van Loan, C.F. (2013). *Matrix computations*, 4th ed. Johns Hopkins.
- Higham, N.J. (2002). *Accuracy and stability of numerical algorithms*, 2nd ed. SIAM.
- Hutchinson, M.F. (1990). A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Commun. Stat. Simul. Comput.* 19, 433–450.
- Koch, K.-R. (1999). *Parameter estimation and hypothesis testing in linear models*, 2nd ed. Springer.
- Koch, K.-R., Kusche, J. (2002). Regularization of geopotential determination from satellite data by variance components. *J. Geod.* 76, 259–268.
- Kusche, J. (2003). A Monte-Carlo technique for weight estimation in satellite geodesy. *J. Geod.* 76, 641–652.
- Kvas, A. (2019). *Estimation of high-frequency mass variations from satellite data in near real-time*. Doctoral thesis, Graz University of Technology. §2.2–2.3, §3.3.
- Kvas, A., Mayer-Gürr, T. (2019). Time-variable gravity field estimation from GRACE with stochastic modelling of temporal variations. *J. Geod.* 93, 1–16.
- Pope, A.J. (1976). *The statistics of residuals and the detection of outliers*. NOAA Tech. Rep. NOS 65 NGS 1.
- Rao, C.R. (1971). Estimation of variance and covariance components — MINQUE theory. *J. Multivariate Anal.* 1, 257–275.
- Teunissen, P.J.G. (2000). *Adjustment theory*. Delft University Press.
- Teunissen, P.J.G., Amiri-Simkooei, A.R. (2008). Least-squares variance component estimation. *J. Geod.* 82, 65–82.
- Yang, Y. (1994). Robust estimation for dependent observations. *Manuscr. Geod.* 19, 10–17.
- MathWorks (2025). *MATLAB Coding Guidelines* v1.0; `https://github.com/matlab/rules/blob/main/matlab-coding-standards.md`.
- Weigelt, M. (2026). `referenceSystem`: `CODING_GUIDE.md`, `VALIDATION_GUIDE.md`, `DOCUMENTATION_GUIDE.md` (`https://github.com/matweigelt/referenceSystem`).